

Curso: .NET 10 e System Design para processamento de arquivos em alto volume

Oct 5, 2026 · @Daniel Freire

Como usar este curso

Você vai sair daqui sabendo projetar e implementar, em .NET 10 (a versão LTS atual), um pipeline que recebe milhares de arquivos por dia, processa milhões de linhas e grava tudo no banco sem estourar memória, sem duplicar dados e sem perder registros quando algo falha.

O curso é **top-down**: começa pelo problema de negócio e pelas métricas, desce para a arquitetura, depois para cada componente e só então para o código. A ideia é que você sempre saiba *por que* uma linha de código existe antes de ver *como* ela é escrita. Cada módulo responde a uma pergunta de design e termina com um exercício que vira peça do projeto final.

Pré-requisitos

O curso assume que você já domina C# básico e intermediário: classes, interfaces, generics, LINQ, `async/await`, injeção de dependência e o básico de EF Core. Nada disso é reexplicado. O foco está no que muda quando o volume cresce: memória, I/O, concorrência, banco e operação.

Exemplo condutor

Todo o curso usa o mesmo caso: arquivos de retorno bancário em layout posicional (CNAB 240/400). Eles são perfeitos para estudar alto volume porque têm linhas de largura fixa, chegam em lotes, precisam ser processados exatamente uma vez e alimentam tabelas grandes. Tudo que você aprender vale igualmente para CSV, JSON Lines, XML em streaming ou logs.

Stack usada

- .NET 10 + C# 14, Worker Service (`BackgroundService`) e Generic Host
- PostgreSQL com Npgsql e EF Core 10 (com notas para SQL Server onde a técnica muda)
- Armazenamento de objetos (S3 ou compatível) e uma fila (SQS, RabbitMQ ou Azure Service Bus — os conceitos são iguais)

- OpenTelemetry, BenchmarkDotNet e as ferramentas `dotnet-counters` / `dotnet-trace`

Mapa dos módulos

Módulo	Pergunta que responde	O que você entrega
1. O problema	Quanto volume, com que prazo, com que garantias?	Uma ficha de requisitos com números
2. Arquitetura	Quais peças existem e como conversam?	Um diagrama de referência
3. Runtime	O que o .NET faz por mim e onde eu atrapalho?	Configuração de GC e host
4. I/O de arquivos	Como ler 2 GB sem carregar 2 GB?	Um parser em streaming sem alocação
5. Concorrência	Como paralelizar sem afogar o banco?	Um pipeline com Channels e back-pressure
6. Escrita em massa	Como inserir 1 milhão de linhas em segundos?	Um writer com COPY e upsert
7. Leitura em massa	Como ler tabelas grandes sem travar nada?	Consultas em streaming e keyset
8. Confiabilidade	O que acontece quando algo quebra no meio?	Idempotência, outbox e DLQ
9. Observabilidade	Como sei que está rápido e saudável?	Métricas, traces e benchmarks
10. Escala	Como crescer de 1 para N workers?	Deploy com autoscaling por fila
Projeto final	Tudo junto funciona?	O pipeline completo, testado sob carga

Sugestão de ritmo: um módulo por semana, lendo a teoria primeiro e fazendo o exercício no fim de semana. Os módulos 4, 5 e 6 são o coração técnico; vale gastar mais tempo neles.

Módulo 1 — O problema: anatomia de um pipeline de alto volume

Antes de escrever uma linha de código, você precisa de três números: quantas linhas por segundo o sistema precisa sustentar no pico, em quanto tempo um arquivo precisa estar processado e qual garantia de entrega o negócio exige. Sem eles, toda decisão de arquitetura é chute.

1.1 O que é "alto volume" de verdade

"Alto volume" não é um número absoluto; é quando o jeito ingênuo para de funcionar. O jeito ingênuo é: `File.ReadAllLines`, um `foreach`, um `SaveChanges` por linha. Ele quebra em três pontos previsíveis:

1. **Memória:** o arquivo inteiro vira um `string[]` no heap. Um arquivo de 500 MB pode ocupar mais de 1 GB em memória, porque `string` em .NET é UTF-16 (2 bytes por caractere) e cada linha tem um cabeçalho de objeto.
2. **Banco:** um `INSERT` por linha significa uma ida e volta de rede por linha. Com 1 ms de latência, 1 milhão de linhas custam 1.000 segundos só de espera, mesmo que o banco esteja ocioso.
3. **Falha:** se o processo cai na linha 800 mil, você não sabe o que foi gravado. Reprocessar duplica; não reprocessar perde.

O curso inteiro é sobre resolver esses três pontos de forma sistemática.

1.2 Requisitos funcionais e não funcionais

Requisitos funcionais dizem o *que* o sistema faz: ler o arquivo, validar o layout, converter cada linha em um evento de negócio (liquidação, baixa, rejeição), gravar e notificar.

Requisitos não funcionais dizem *como bem* ele faz, e são eles que definem a arquitetura.

Para um pipeline de arquivos, os que importam são:

Requisito	Pergunta para fazer ao negócio	Por que muda o design
Vazão (throughput)	Quantas linhas por segundo no pico?	Define número de workers e técnica de escrita no banco
Latência ponta a ponta	Em quanto tempo após a chegada o dado precisa estar disponível?	Define se dá para fazer em lote ou se precisa de streaming contínuo
Garantia de entrega	Pode perder? Pode duplicar?	Define idempotência, transações e checkpoint
Ordenação	A ordem das linhas ou dos arquivos importa?	Limita o paralelismo e exige particionamento

Requisito	Pergunta para fazer ao negócio	Por que muda o design
Tamanho dos arquivos	Qual o maior arquivo possível, não o médio?	Define se dá para processar em memória e como dividir
Retenção e auditoria	Por quanto tempo guardar o arquivo original?	Define o armazenamento e a rastreabilidade

Uma regra prática: projete para o **pior arquivo** e para a **hora de pico**, não para a média. Arquivos bancários costumam chegar concentrados em janelas curtas (início da manhã, fim do dia), e a média diária esconde isso.

1.3 Contas de guardanapo (back-of-the-envelope)

Vamos dimensionar um cenário de exemplo. Os números são hipotéticos, mas o método é o que importa.

- 5.000 arquivos por dia, média de 40.000 linhas, maior arquivo com 3 milhões de linhas
- 60% dos arquivos chegam numa janela de 2 horas
- Cada arquivo precisa estar processado em até 15 minutos após a chegada

Total diário: $5.000 \times 40.000 = 200$ milhões de linhas. Na janela de pico: 60% de 200 milhões = 120 milhões de linhas em 7.200 segundos, ou seja, cerca de **16.700 linhas por segundo** sustentadas. Com margem de segurança de $2\times$, o alvo de projeto vira algo em torno de 35.000 linhas/s.

O maior arquivo (3 milhões de linhas) precisa terminar em 15 minutos, então um único arquivo exige pelo menos 3.300 linhas/s. Isso diz que um arquivo grande não pode depender de um caminho lento, e talvez precise ser dividido em pedaços.

Agora o banco: se cada linha gera uma linha numa tabela de 300 bytes, 200 milhões de linhas por dia são 60 GB por dia de dados brutos, sem contar índices. Em um ano, mais de 20 TB. Essa conta, feita no primeiro dia, já obriga a pensar em **particionamento de tabela** e **política de retenção** (Módulo 7).

1.4 Lei de Little: quantos workers eu preciso?

A Lei de Little relaciona três grandezas de qualquer sistema estável com fila:

$$L = \lambda \cdot W$$

L é o número médio de itens em processamento ao mesmo tempo, λ é a taxa de chegada (itens por segundo) e W é o tempo médio que cada item passa no sistema. Ela é a ferramenta mais útil para dimensionar concorrência.

Exemplo: se o alvo é 35.000 linhas/s e cada lote de 5.000 linhas leva 0,5 s para ser gravado, a taxa é 7 lotes por segundo e cada lote fica 0,5 s no sistema. Logo $L = 7 \times 0,5 = 3,5$. Você precisa de pelo menos 4 escritores concorrentes. Se a escrita fica mais lenta (W sobe para 1 s), precisa de 7. A lei mostra que **latência e concorrência são intercambiáveis**: quando um sobe, o outro precisa subir para manter a vazão.

1.5 Onde está o gargalo

Todo pipeline tem um estágio mais lento, e a vazão total é a vazão desse estágio. Existem três tipos de gargalo, e cada um tem remédio diferente:

- **I/O de arquivo ou rede** (baixar do S3, ler disco): o processo espera e a CPU fica ociosa. Remédio: `async`, buffers maiores, ler e processar em paralelo.
- **CPU** (parsing, validação, conversão): a CPU fica em 100% em um núcleo ou em todos. Remédio: código sem alocação (Módulo 3 e 4) e paralelismo até o número de núcleos.
- **Banco** (escrita, locks, índices): a CPU do worker fica ociosa esperando o banco. Remédio: lotes maiores, `COPY`, menos índices na tabela de chegada, e **limitar** a concorrência para não piorar o banco (Módulo 5 e 6).

Em pipelines de arquivo, o gargalo quase sempre é o banco. Por isso o design gira em torno de proteger o banco: escrever em lotes, com concorrência limitada e com back-pressure, que é a capacidade de o estágio lento frear o estágio rápido em vez de deixar a memória encher.

1.6 Garantias de entrega

Existem três garantias possíveis, e só duas são realistas:

- **At-most-once** (no máximo uma vez): pode perder, nunca duplica. Inaceitável para dado financeiro.
- **At-least-once** (pelo menos uma vez): nunca perde, pode duplicar. É o que filas e retries entregam naturalmente.
- **Exactly-once** (exatamente uma vez): o que o negócio quer. Na prática, se obtém como **at-least-once + idempotência**: o sistema pode tentar várias vezes, mas o efeito no banco acontece uma vez só.

Guarde essa equação; ela volta no Módulo 8 e é a base de todo o design de confiabilidade.

Exercício do Módulo 1

Escreva a ficha de requisitos do seu cenário real (ou use o cenário de exemplo). Preencha: arquivos por dia, linhas por arquivo (média e máximo), janela de pico, SLA por arquivo, garantia de entrega e retenção. Calcule as linhas por segundo de pico, o volume diário em

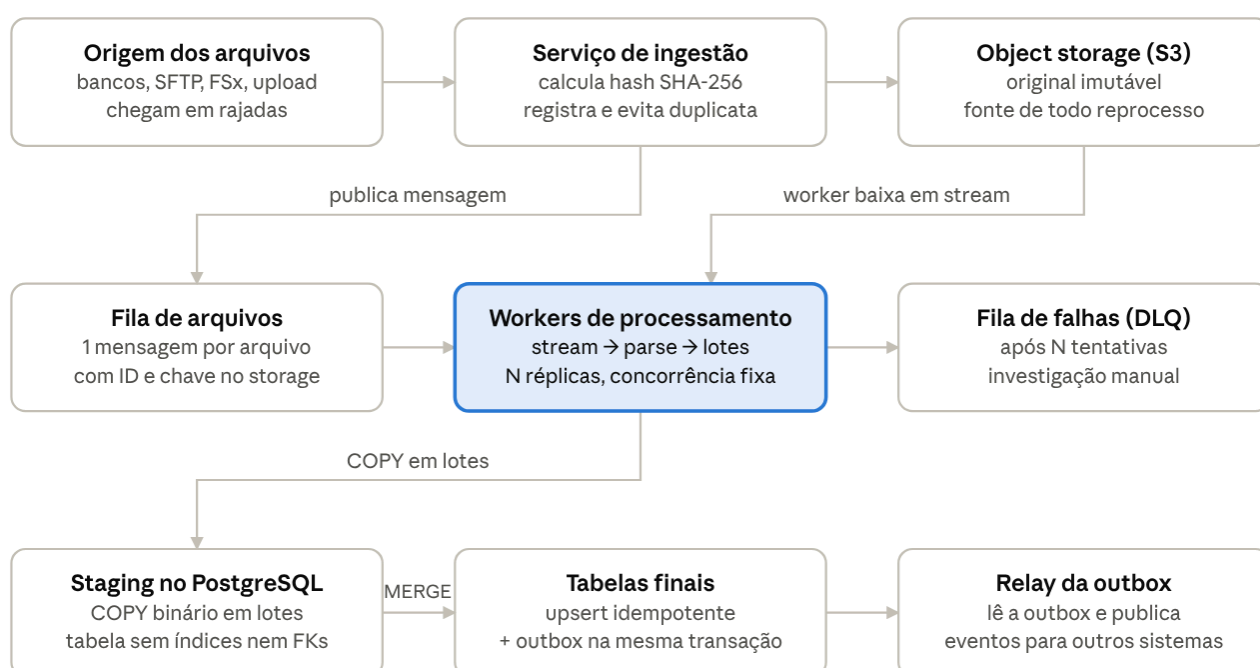
GB e o número de escritores pela Lei de Little. Essa ficha será usada para validar o projeto final.

Módulo 2 — System design: a arquitetura de referência

A arquitetura que funciona para alto volume separa **receber** de **processar**: o arquivo é guardado intacto e registrado em segundos, e o processamento pesado acontece depois, em workers que consomem uma fila, escrevem em lotes numa área de staging e consolidam nas tabelas finais com uma operação idempotente.

O arquivo entra uma vez, é processado em lotes e vira eventos pela outbox

Arquitetura de referência · o worker em destaque é o foco dos Módulos 4 a 7



arquitetura de referência · 9 componentes, 3 camadas

Leia de cima para baixo: a primeira linha recebe e guarda, a segunda distribui o trabalho, a terceira grava e propaga. Cada seta é um ponto onde algo pode falhar, e cada componente foi escolhido para que essa falha seja recuperável.

2.1 Os seis princípios

1. **Separe ingestão de processamento.** Quem recebe o arquivo não o processa. Assim, uma rajada de 2.000 arquivos é aceita em segundos e drenada no ritmo que o banco aguenta. Sem essa separação, a rajada vira timeout no remetente.
2. **O arquivo original é imutável e fica guardado.** Tudo pode ser reprocessado a partir dele. Isso transforma qualquer bug de parsing num problema de "corrigir e reprocessar", e não de "recuperar dado perdido".

3. **A unidade de trabalho é explícita.** Normalmente é o arquivo; para arquivos gigantes, é um pedaço (chunk) do arquivo. Toda mensagem na fila aponta para uma unidade, nunca carrega o conteúdo.
4. **Workers são stateless.** Um worker pode morrer a qualquer momento e outro assume. Todo estado relevante (status do arquivo, até onde foi processado) mora no banco.
5. **O banco é protegido.** Escrita em lote, concorrência limitada e back-pressure. O banco é o recurso mais caro e mais difícil de escalar; o resto do sistema se adapta a ele.
6. **Todo passo é idempotente.** Executar duas vezes produz o mesmo resultado que executar uma. É isso que torna retries seguros.

2.2 Cada componente, em detalhe

Serviço de ingestão. Recebe o arquivo (upload HTTP, leitura de SFTP, polling de um compartilhamento SMB/FSx), calcula o SHA-256 enquanto grava no storage e insere uma linha na tabela `arquivo` com uma restrição `UNIQUE` sobre o par (cliente, hash). Se o mesmo arquivo chegar duas vezes, o `INSERT` falha e a duplicata é descartada logo na porta. Só depois de gravar no storage e registrar no banco ele publica a mensagem na fila. Esse serviço é leve: não faz parsing, só move bytes.

Object storage. S3 (ou equivalente) guarda o original com uma chave previsível, por exemplo `retornos/2026/10/05/<hash>.ret`. Ele é barato, durável e permite leitura em stream, o que significa que o worker nunca precisa baixar o arquivo inteiro para o disco antes de começar.

Fila de arquivos. Cada mensagem tem o ID do arquivo e a chave no storage, e nada mais. A fila dá três coisas de graça: distribuição de trabalho entre workers, reentrega automática se um worker morrer (visibility timeout no SQS, ack no RabbitMQ) e uma métrica natural de atraso (tamanho da fila), que será usada para autoscaling no Módulo 10.

Workers de processamento. O coração do curso. Cada worker lê o arquivo em stream, faz parsing linha a linha sem alocar strings, valida, agrupa em lotes de alguns milhares de registros e envia cada lote ao banco com `COPY`. Internamente ele é um pipeline de estágios conectados por `Channel<T>` com capacidade limitada (Módulo 5).

Fila de falhas (DLQ). Mensagens que falham N vezes vão para uma fila separada. Isso evita que um arquivo envenenado (poison message) trave os demais e dá um lugar claro para investigação.

Staging. Uma tabela sem índices secundários nem chaves estrangeiras, que recebe os lotes do jeito mais rápido possível. Cada linha carrega o ID do arquivo e o número da linha original. Separar staging de tabela final é o truque central de desempenho: inserir em tabela sem índice é muito mais rápido, e a consolidação acontece em SQL, dentro do banco, onde é eficiente.

Tabelas finais. Recebem os dados via `INSERT ... ON CONFLICT DO UPDATE` ou `MERGE`, usando uma chave natural (por exemplo, arquivo + número da linha, ou o identificador de negócio do título). Na mesma transação, gravam na tabela `outbox` os eventos de negócio gerados.

Relay da outbox. Um processo que lê a tabela `outbox` e publica os eventos para outros sistemas. Ele resolve o problema clássico de "gravei no banco mas não consegui publicar a mensagem" (Módulo 8).

2.3 O ciclo de vida de um arquivo

O status do arquivo na tabela `arquivo` é a fonte da verdade sobre o que aconteceu com ele. Toda transição é um `UPDATE` condicional (`WHERE status = 'esperado'`), o que impede que dois workers peguem o mesmo arquivo.

Estado	Significado	Quem muda e para onde
RECEBIDO	Gravado no storage e registrado	Ingestão cria; worker muda para <code>PROCESSANDO</code>
PROCESSANDO	Um worker está com ele, com prazo (lease)	Worker muda para <code>CONCLUIDO</code> , <code>REJEITADO</code> ou <code>FALHOU</code>
CONCLUIDO	Todas as linhas consolidadas	Estado final
REJEITADO	Layout inválido; nada foi aplicado	Estado final; vai para análise
FALHOU	Erro técnico após N tentativas	Operação pode voltar para <code>RECEBIDO</code>

O **lease** é um campo `processando_ate` com timestamp. Se o worker morrer, o lease expira e outro worker pode reassumir. Isso cobre o caso em que a fila reentrega a mensagem mas o banco ainda acha que alguém está trabalhando.

A tabela `arquivo` guarda esse ciclo de vida e é usada em quase todos os módulos (e também pela API do curso de Minimal APIs, que lê e grava a mesma tabela). Vale fixá-la desde já:

```
CREATE TABLE arquivo (
  id                uuid          PRIMARY KEY,          -- UUID v7,
  gerado pela ingestão
  sequencia         bigint        GENERATED ALWAYS AS IDENTITY UNIQUE, --
  ordem total, usada em paginação
  cliente_id        uuid          NOT NULL,
  banco             text          NOT NULL,
  nome_original     text          NOT NULL,
  sha256            text          NOT NULL,
```

```

    chave_storage      text          NOT NULL,
    status              text          NOT NULL DEFAULT 'RECEBIDO'
        CHECK (status IN ('RECEBIDO', 'PROCESSANDO', 'CONCLUIDO',
    'REJEITADO', 'FALHOU')),
    tentativas          int           NOT NULL DEFAULT 0,
    worker_id           text          NULL,
    processando_ate     timestamptz  NULL,                -- lease
    linhas_processadas  int           NOT NULL DEFAULT 0,
    linhas_rejeitadas   int           NOT NULL DEFAULT 0,
    motivo              text          NULL,                -- por que foi
    rejeitado ou falhou
    recebido_em         timestamptz  NOT NULL DEFAULT now(),
    data_recebimento    date          NOT NULL,            -- data de
    negócio (fuso de São Paulo), gravada pela ingestão
    CONSTRAINT uq_arquivo_conteudo UNIQUE (cliente_id, sha256)
);

CREATE INDEX ix_arquivo_cliente_sequencia ON arquivo (cliente_id, sequencia
DESC);
CREATE INDEX ix_arquivo_pendentes ON arquivo (recebido_em) WHERE status IN
('RECEBIDO', 'PROCESSANDO');

```

O `UNIQUE (cliente_id, sha256)` é a deduplicação da ingestão: o mesmo conteúdo enviado duas vezes pelo mesmo cliente é recusado, mas dois clientes diferentes podem enviar arquivos idênticos sem conflito. O índice parcial de pendentes atende o job que reenfileira arquivos esquecidos e fica sempre pequeno, porque arquivos concluídos saem dele.

2.4 Arquivos gigantes: dividir para paralelizar

Um arquivo de 3 milhões de linhas processado por um único worker fica limitado à vazão desse worker. A saída é o padrão **fan-out / fan-in**:

1. Um passo de preparação lê o arquivo sem parsear, só contando quebras de linha, e calcula offsets de byte a cada N linhas (por exemplo, a cada 200 mil). Em layout de largura fixa é ainda mais simples: o offset da linha k é $k \times (\text{tamanho da linha} + \text{quebra})$.
2. Ele cria registros na tabela `arquivo_chunk` (arquivo, início, fim, status) e publica uma mensagem por chunk.
3. Workers processam chunks em paralelo, lendo apenas o intervalo de bytes deles (o S3 aceita leitura por range).
4. Quando o último chunk termina (contagem de chunks concluídos igual ao total, verificada em transação), o arquivo passa para `CONCLUIDO`.

Atenção: arquivos CNAB têm header e trailer com totais de controle. A validação do trailer (quantidade de registros, somatório de valores) só pode ser feita no fan-in, comparando com a soma dos chunks.

2.5 Alternativas e quando escolher cada uma

Decisão	Opção A	Opção B	Escolha A quando	Escolha B quando
Distribuição de trabalho	Fila gerenciada (SQS, Service Bus)	Tabela como fila (<code>FOR UPDATE SKIP LOCKED</code>)	Muitos consumidores, autoscaling, vários serviços	Volume moderado e você quer menos infraestrutura
Chegada do arquivo	Evento (notificação do S3)	Polling de pasta	O storage emite eventos	Origem legada (SMB, SFTP) sem eventos
Escrita	Staging + MERGE	Upsert direto na final	Lotes grandes, tabela final com muitos índices	Lotes pequenos e baixa latência
Divisão	Arquivo inteiro por worker	Chunks	Arquivos até algumas centenas de milhares de linhas	Arquivos de milhões de linhas com SLA curto

A tabela como fila merece atenção porque é subestimada. No PostgreSQL, `SELECT ... FOR UPDATE SKIP LOCKED LIMIT 1` permite que vários workers peguem itens diferentes sem bloquear uns aos outros. Para alguns milhares de arquivos por dia, isso elimina uma peça de infraestrutura inteira e deixa status e fila na mesma transação.

Exercício do Módulo 2

Desenhe a arquitetura para o seu cenário da ficha do Módulo 1. Para cada seta, escreva o que acontece se o componente de destino estiver fora do ar por 10 minutos: o que se perde, o que se acumula e como o sistema se recupera. Se alguma resposta for "perde dados", o design ainda não está pronto.

Módulo 3 — Runtime .NET 10 e C# 14: o que importa para vazão

Em processamento de alto volume, o maior inimigo dentro do processo é a **alocação**: cada objeto criado por linha vira trabalho para o Garbage Collector, e milhões de linhas viram milhões de objetos. O .NET 10 entrega as ferramentas para processar sem alocar

(`Span<T>` , pools, parsing UTF-8), e este módulo ensina a usá-las e a configurar o runtime para um worker.

Nota de versão: o .NET 10 é a versão LTS atual, com patches mensais (10.0.12 saiu em setembro de 2026). O .NET 11 está em Release Candidate com licença go-live e tem lançamento previsto para novembro de 2026, junto com o C# 15 (que traz union types). Para um sistema de produção de longa duração, fique no .NET 10; tudo neste curso funciona igual no 11.

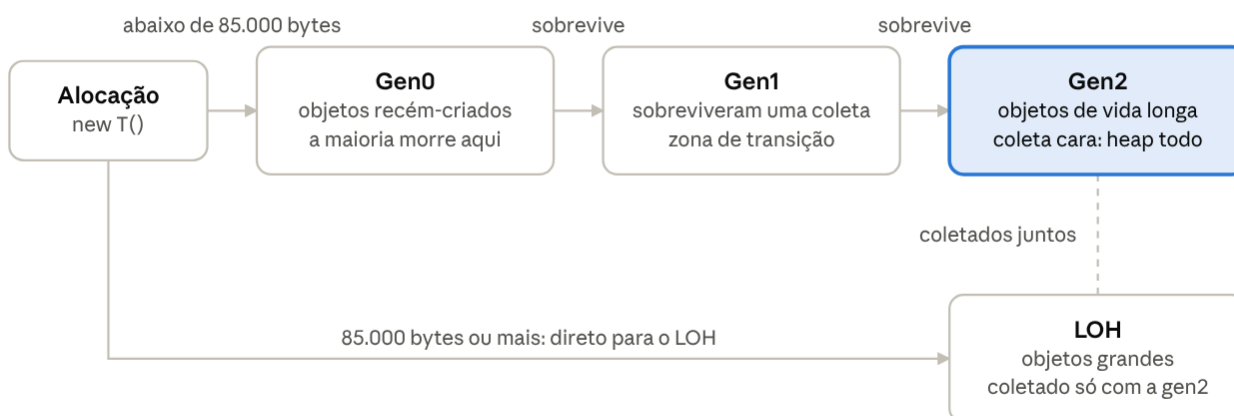
3.1 Como o GC funciona e por que você deve se importar

O GC do .NET é **geracional**. Objetos novos nascem na geração 0 (gen0). Uma coleta de gen0 é barata porque a maioria dos objetos já morreu. Quem sobrevive sobe para gen1 e depois para gen2. Coletas de gen2 percorrem o heap inteiro e são caras.

Há também o **LOH** (Large Object Heap): objetos com 85.000 bytes ou mais (por exemplo, um `byte[]` de 100 KB ou uma `List<T>` grande) vão direto para ele e só são coletados junto com gen2. Alocar buffers grandes repetidamente é a forma mais rápida de causar coletas completas.

Objeto que morre jovem é barato; o caro é sobreviver ou ser grande

Gerações do GC do .NET · em destaque, a coleta que percorre o heap inteiro



GC geracional · gen0, gen1, gen2 e LOH

O objetivo do hot path é que nada chegue à direita do desenho: objetos que morrem na gen0 custam quase nada, enquanto buffers grandes vão direto para o LOH, que só é limpo pela coleta cara.

O cálculo que importa: se o parser cria 10 strings por linha e o arquivo tem 3 milhões de linhas, são 30 milhões de objetos. Mesmo que cada coleta de gen0 seja rápida, o custo acumulado aparece como CPU gasta em GC e como picos de latência. O objetivo do Hot Path (o código que roda por linha) é **zero alocação por linha**.

3.2 Configurando o GC para um worker

Existem dois modos de GC:

- **Workstation GC:** um heap, otimizado para baixa latência em apps interativos. Padrão em apps de console e workers.
- **Server GC:** um heap por núcleo, coletas paralelas, muito mais vazão. Padrão em ASP.NET Core.

Para um worker de processamento em lote, Server GC é quase sempre a escolha certa. Desde o .NET 9, o Server GC vem com **DATAS** (Dynamic Adaptation To Application Sizes) ligado, que ajusta o número de heaps conforme a carga para economizar memória. Em containers com memória apertada isso é ótimo; num worker dedicado que precisa de vazão máxima, vale medir com e sem.

```
<PropertyGroup>
  <TargetFramework>net10.0</TargetFramework>
  <Nullable>enable</Nullable>
  <ServerGarbageCollector>true</ServerGarbageCollector>
  <ConcurrentGarbageCollection>true</ConcurrentGarbageCollection>
  <!-- 0 desliga DATAS; remova a linha para manter o padrão -->
  <GarbageCollectionAdaptationMode>0</GarbageCollectionAdaptationMode>
</PropertyGroup>
```

Em containers, o runtime lê o limite de memória do cgroup e, por padrão, limita o heap a 75% dele. Se o pod tem 2 GiB, o heap gerenciado para em cerca de 1,5 GiB e o resto fica para código nativo, buffers do Npgsql e pilhas de threads. Você pode ajustar com a variável `DOTNET_GCHeapHardLimitPercent` (valor em hexadecimal: `0x4B` = 75).

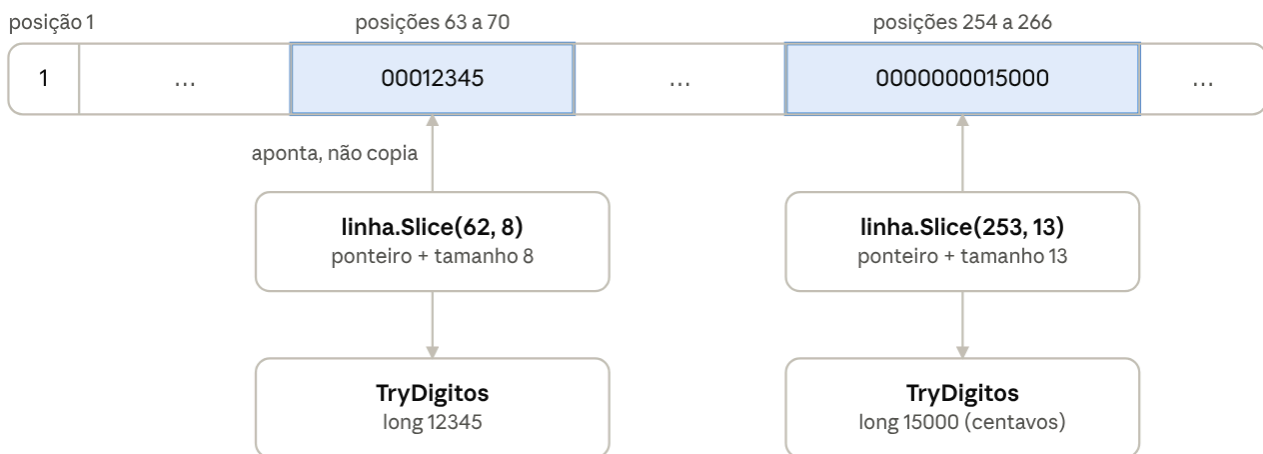
Regra prática para Kubernetes: defina `requests` e `limits` de memória iguais para o worker. Isso torna o comportamento do GC previsível e evita que o pod seja morto por OOM quando o nó fica cheio.

3.3 Span<T> e ReadOnlySpan<T>: uma janela sobre memória existente

Um `Span<T>` é um ponteiro mais um tamanho. Ele não copia nada: aponta para um pedaço de um array, de memória nativa ou da pilha. Fatiar (`slice`) é gratuito. Isso permite olhar para os bytes 63 a 70 de uma linha sem criar uma string.

Um Span é uma janela sobre bytes que já existem: fatiar não copia nada

Uma linha CNAB de 400 bytes no buffer · fora de escala



Nenhuma string é criada: o buffer continua sendo o único dono dos bytes

Span sobre uma linha CNAB · 2 fatias, nenhuma cópia

Os dois spans apontam para o mesmo array; só o `TryDigits` lê os bytes, e o resultado é um `long` na pilha.

Regras que você precisa decorar:

- `Span<T>` é um `ref struct`: vive só na pilha. Não pode ser campo de classe, não pode ser capturado em lambda e **não pode atravessar um `await`**.
- Quando você precisa guardar a referência ou usá-la em código assíncrono, use `Memory<T>` / `ReadOnlyMemory<T>`, e chame `.Span` dentro do trecho síncrono.
- O padrão resultante é: I/O assíncrono trabalha com `Memory<byte>`; parsing síncrono trabalha com `ReadOnlySpan<byte>`.

No C# 14, spans ganharam conversões implícitas de primeira classe: um `byte[]` ou uma `string` convertem automaticamente para o span correspondente em mais situações, incluindo chamadas de métodos de extensão. Na prática, você escreve menos `.AsSpan()`.

3.4 Trabalhando em UTF-8 (ou Latin-1), não em string

Arquivos chegam como bytes. Convertê-los para `string` custa duas vezes: decodificar e alocar o dobro do tamanho (UTF-16). Para formatos posicionais, o melhor é **parsear direto dos bytes**. Os tipos numéricos do .NET aceitam UTF-8:

`int.TryParse(ReadOnlySpan<byte>, ...)`, `long.TryParse`, `decimal.TryParse` e também `System.Text.Utf8Parser`.

Arquivos bancários brasileiros costumam usar ISO-8859-1 (Latin-1), não UTF-8. Para campos numéricos isso não muda nada (dígitos ASCII são iguais nos dois). Para campos de texto que você realmente precisa guardar (nome do pagador), decodifique só aquele campo com `Encoding.Latin1.GetString(span)`, e só quando necessário.

Um parser de registro sem alocação fica assim (posições ilustrativas; cada banco tem as suas):

```
using System.Runtime.CompilerServices;

public readonly record struct RegistroRetorno(
    long NossoNumero,
    byte CodigoOcorrencia,
    DateOnly DataOcorrencia,
    long ValorPagoCentavos,
    int NumeroLinha);

public static class Cnab400Parser
{
    public const int TamanhoLinha = 400;

    public static bool TryParseDetalhe(ReadOnlySpan<byte> linha, int
                                numeroLinha,
                                out RegistroRetorno registro, out
                                string? erro)
    {
        registro = default;
        erro = null;

        if (linha.Length != TamanhoLinha) { erro = "tamanho inválido"; return
false; }
        if (linha[0] != (byte)'1')          { erro = "não é detalhe";      return
false; }

        // Posições do layout são 1-based e inclusivas: 63 a 70 vira
Slice(62, 8)
        if (!TryDigitos(linha.Slice(62, 8), out long nossoNumero)) { erro =
"nosso número"; return false; }
        if (!TryDigitos(linha.Slice(108, 2), out long ocorrencia)) { erro =
"ocorrência"; return false; }
        if (!TryDataDdMmAa(linha.Slice(110, 6), out var data))      { erro =
"data"; return false; }
        if (!TryDigitos(linha.Slice(253, 13), out long valor))      { erro =
"valor pago"; return false; }

        registro = new RegistroRetorno(nossoNumero, (byte)ocorrencia, data,
```

```

valor, numeroLinha);
    return true;
}

// Trailer (registro '9'): quantidade de registros e valor total, usados
na reconciliação (Módulo 8)
public static bool TryParseTrailer(ReadOnlySpan<byte> linha,
                                   out int quantidadeRegistros, out long
valorTotalCentavos)
{
    quantidadeRegistros = 0;
    valorTotalCentavos = 0;
    if (linha.Length != TamanhoLinha || linha[0] != (byte)'9') return
false;
    if (!TryDigitos(linha.Slice(17, 8), out long quantidade)) return
false;
    if (!TryDigitos(linha.Slice(25, 14), out valorTotalCentavos)) return
false;
    quantidadeRegistros = (int)quantidade;
    return true;
}

[MethodImpl(MethodImplOptions.AggressiveInlining)]
private static bool TryDigitos(ReadOnlySpan<byte> campo, out long valor)
{
    valor = 0;
    foreach (byte b in campo)
    {
        uint d = (uint)(b - (byte)'0');
        if (d > 9) return false;
        valor = valor * 10 + d;
    }
    return true;
}

private static bool TryDataDdMmAa(ReadOnlySpan<byte> campo, out DateOnly
data)
{
    data = default;
    if (!TryDigitos(campo[..2], out long dd) ||
        !TryDigitos(campo.Slice(2, 2), out long mm) ||
        !TryDigitos(campo.Slice(4, 2), out long aa)) return false;
    if (mm is < 1 or > 12 || dd < 1 || dd > DateTime.DaysInMonth(2000 +
(int)aa, (int)mm)) return false;

```

```

        data = new DateOnly(2000 + (int)aa, (int)mm, (int)dd);
        return true;
    }
}

```

Repare nos detalhes: o registro é um `readonly record struct` (sem alocação no heap); o valor monetário fica em centavos como `long` (sem `decimal` nem `double` no hot path); o erro é uma string literal, que não aloca; e o loop de dígitos usa o truque `(uint)(b - '0') > 9` para validar com uma única comparação.

3.5 Pools: reutilize em vez de alocar

Buffers grandes devem ser alugados, não criados. `ArrayPool<T>.Shared` mantém arrays reutilizáveis por tamanho:

```

byte[] buffer = ArrayPool<byte>.Shared.Rent(256 * 1024);
try
{
    int lidos = await stream.ReadAsync(buffer.AsMemory(), ct);
    Processar(buffer.AsSpan(0, lidos));
}
finally
{
    ArrayPool<byte>.Shared.Return(buffer);
}

```

Três cuidados: o array devolvido pode ser **maior** que o pedido (use sempre o tamanho que você escreveu, não `buffer.Length`); **nunca use** o array depois de devolver; e devolva sempre num `finally`. Para listas de objetos que você monta e descarta por lote (como o lote de registros que vai para o banco), reutilize a mesma `List<T>` chamando `Clear()`, que mantém a capacidade interna.

Para `MemoryStream` em alto volume, use o pacote

`Microsoft.IO.RecyclableMemoryStream`, que reaproveita blocos e evita o LOH.

3.6 Buscas vetorizadas com SearchValues

O pipeline precisa achar quebras de linha milhões de vezes. `span.IndexOf((byte)'\n')` já é vetorizado (usa SIMD, compara 16 a 64 bytes por instrução). Quando você procura um conjunto de valores, por exemplo `\r` ou `\n`, crie uma vez um `SearchValues<byte>`:

```
private static readonly SearchValues<byte> QuebraDeLinha =  
SearchValues.Create("\r\n"u8);  
  
int pos = span.IndexOfAny(QuebraDeLinha);
```

O sufixo `u8` cria um literal UTF-8 em tempo de compilação, sem alocação.

3.7 O que o C# 14 e o JIT do .NET 10 trazem de útil aqui

- **Extension members:** além de métodos, agora dá para declarar propriedades e membros estáticos de extensão num bloco `extension`. Útil para criar APIs fluentes sobre `ReadOnlySpan<byte>` (por exemplo, `linha.Campo(63, 70)`).
- **Palavra-chave `field`:** acessa o campo gerado de uma auto-property, permitindo validação no `set` sem declarar o campo à mão. Útil em classes de options.
- **Atribuição null-condicional** (`cliente?.Status = ...`): menos `if` em código de mapeamento.
- **JIT:** o .NET 10 ampliou a análise de escape, permitindo alocar na pilha objetos pequenos e arrays de tamanho fixo que não escapam do método, além de mais desvirtualização. Dynamic PGO (otimização guiada por perfil em tempo de execução) segue ligado por padrão. Você ganha isso de graça ao atualizar, mas não conte com isso no lugar de escrever código sem alocação.

3.8 O host: BackgroundService bem feito

Todo worker roda dentro do Generic Host, que entrega DI, configuração, logging e, principalmente, **desligamento gracioso**. Quando o Kubernetes manda SIGTERM, o host cancela o `CancellationToken` do `ExecuteAsync` e espera até `ShutdownTimeout` antes de encerrar.

```

var builder = Host.CreateApplicationBuilder(args);

builder.Services.Configure<HostOptions>(o =>
{
    o.ShutdownTimeout = TimeSpan.FromSeconds(45);
    o.BackgroundServiceExceptionBehavior =
BackgroundServiceExceptionBehavior.StopHost;
});

builder.Services
    .AddOptions<ProcessamentoOptions>()
    .Bind(builder.Configuration.GetSection("Processamento"))
    .ValidateDataAnnotations()
    .ValidateOnStart();

builder.Services.AddHostedService<ProcessadorDeArquivosWorker>();

await builder.Build().RunAsync();

```

Decisões importantes nesse trecho: `StopHost` faz o processo morrer se o worker lançar exceção não tratada (é melhor o pod reiniciar do que ficar vivo sem trabalhar); `ValidateOnStart` faz configuração inválida falhar no boot, não às 3 da manhã; e o `ShutdownTimeout` deve ser menor que o `terminationGracePeriodSeconds` do pod, para dar tempo de terminar o lote atual.

No `ExecuteAsync`, não faça trabalho síncrono pesado antes do primeiro `await`: o host só termina de iniciar quando o `ExecuteAsync` devolve o controle. Comece com `await Task.Yield()` se precisar.

Exercício do Módulo 3

Crie um projeto com BenchmarkDotNet e compare duas versões de parsing de 100 mil linhas: uma com `string.Substring` + `long.Parse`, outra com o parser em span acima. Use o atributo `[MemoryDiagnoser]` e anote a diferença de tempo e de bytes alocados. Espere uma redução de alocação de ordens de grandeza; o tempo também deve cair várias vezes.

Módulo 4 — I/O de arquivos de alta performance

O objetivo deste módulo é ler um arquivo de qualquer tamanho com **memória constante**: um arquivo de 10 MB e um de 10 GB devem usar praticamente a mesma quantidade de RAM. A ferramenta para isso é `System.IO.Pipelines`, combinada com o parser em span do Módulo 3.

4.1 Os três níveis de leitura

Abordagem	Memória	Alocação por linha	Quando usar
<code>File.ReadAllLines</code>	Proporcional ao arquivo (cerca de 2× o tamanho)	Uma string por linha, todas vivas ao mesmo tempo	Arquivos de configuração, nunca dados
<code>StreamReader.ReadLineAsync</code>	Constante	Uma string por linha (morre logo)	Volume moderado, protótipos
<code>PipeReader</code> + <code>SequenceReader</code>	Constante	Zero	Alto volume, hot path

O `StreamReader` é um meio-termo honesto e é muito mais simples. Se o seu volume é de alguns milhares de linhas por segundo, ele basta. O `PipeReader` é para quando você precisa de dezenas de milhares de linhas por segundo por núcleo, ou quando o GC aparece nas métricas.

4.2 Abrindo arquivos do jeito certo

Para arquivos locais (ou num volume montado como FSx/SMB), abra o `FileStream` com opções explícitas:

```
var opcoes = new FileStreamOptions
{
    Mode = FileMode.Open,
    Access = FileAccess.Read,
    Share = FileShare.Read,
    Options = FileOptions.Asynchronous | FileOptions.SequentialScan,
    BufferSize = 0 // o PipeReader já faz buffering; evita copiar duas vezes
};
await using var arquivo = new FileStream(caminho, opcoes);
```

`SequentialScan` avisa o sistema operacional de que a leitura é do início ao fim, o que melhora o read-ahead. `BufferSize = 0` desliga o buffer interno do `FileStream`, porque o `PipeReader` terá o seu.

Para ler um pedaço específico (um chunk do fan-out do Módulo 2), use a API `RandomAccess`, que lê por offset sem mover posição compartilhada e é segura entre threads:

```
using SafeFileHandle handle = File.OpenHandle(caminho, FileMode.Open,
FileAccess.Read,
options:
FileOptions.Asynchronous);
int lidos = await RandomAccess.ReadAsync(handle, buffer.AsMemory(),
fileOffset: inicio, ct);
```

4.3 System.IO.Pipelines passo a passo

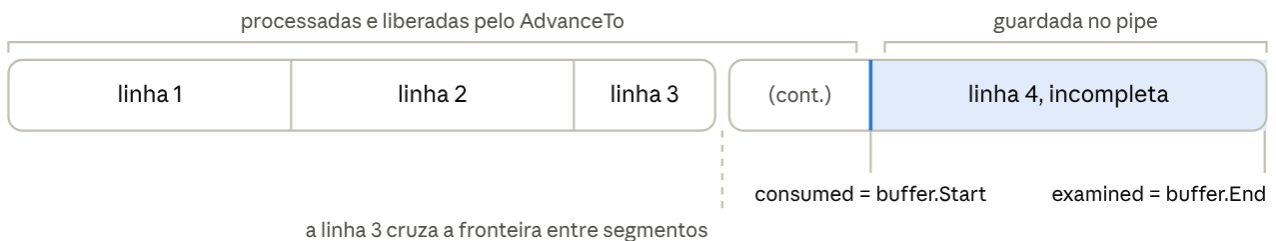
Um **Pipe** resolve o problema mais chato da leitura em blocos: uma linha que começa no fim de um bloco e termina no começo do próximo. Sem Pipelines, você teria que copiar o resto para um buffer auxiliar, gerenciar índices e crescer o buffer. Com Pipelines, isso é automático.

Os conceitos:

- **PipeReader.ReadAsync** devolve um **ReadResult** com um **ReadOnlySequence<byte>**: uma sequência de um ou mais segmentos de memória encadeados. Ela representa tudo que já foi lido e ainda não foi consumido.
- **SequenceReader<byte>** é um cursor sobre essa sequência, com métodos como **TryReadTo(out linha, (byte)'\n')**, que encontra o delimitador mesmo atravessando segmentos.
- **AdvanceTo(consumed, examined)** é o contrato mais importante. **consumed** diz até onde você terminou de usar os dados (pode ser liberado). **examined** diz até onde você olhou. Se você examinou tudo e não achou uma linha completa, o próximo **ReadAsync** espera chegar **mais** dado em vez de devolver o mesmo buffer de novo. Errar isso causa loop infinito ou leitura travada.
- **IsCompleted** indica que a fonte acabou. O que sobrou no buffer é a última linha sem quebra no final, ou lixo.

AdvanceTo devolve o que já foi usado e espera o resto da linha incompleta

Um ReadOnlySequence com dois segmentos · fora de escala

**Próximo ReadAsync: só volta quando chega mais dado, a partir da linha 4**

AdvanceTo(consumed, examined) · duas leituras seguidas

Tudo à esquerda do `consumed` volta para o pool de memória; a linha 4 fica guardada no pipe, e o `ReadAsync` seguinte só retorna quando o resto dela chegar.

O leitor completo, que lê, parseia e entrega os registros para o próximo estágio por um `ChannelWriter` (o Módulo 5 explica o canal):

```
using System Buffers;
using System.IO.Pipelines;
using System.Threading.Channels;

// O que o leitor informa sobre o arquivo, além dos registros válidos.
// A implementação concreta (ContextoDoArquivo) aparece no Módulo 8, na
reconciliação.
public interface IContextoDeLeitura
{
    void RegistrarErro(int numeroLinha, string motivo);
    void RegistrarTrailer(int quantidadeRegistros, long valorTotalCentavos);
    void RegistrarFimDaLeitura(int linhasLidas);
}

public sealed class LeitorCnab400
{
    public async Task LerAsync(Stream origem, ChannelWriter<RegistroRetorno>
saida,
                                IContextoDeLeitura contexto, CancellationToken
ct)
    {
        var reader = PipeReader.Create(origem,
            new StreamPipeReaderOptions(bufferSize: 128 * 1024,
minimumReadSize: 32 * 1024));
```

```

var pendentes = new List<RegistroRetorno>(capacity: 2048);
int numeroLinha = 0;

try
{
    while (true)
    {
        ReadResult resultado = await reader.ReadAsync(ct);
        ReadOnlySequence<byte> buffer = resultado.Buffer;

        // Parte síncrona: aqui pode usar Span à vontade
        numeroLinha = ProcessarBloco(ref buffer,
resultado.IsCompleted,
                                numeroLinha, pendentes,
contexto);

        // consumido = até a última linha completa; examinado = tudo
        reader.AdvanceTo(buffer.Start, buffer.End);

        // Parte assíncrona: se o canal estiver cheio, esperamos
(back-pressure)
        foreach (var registro in pendentes)
            await saida.WriteAsync(registro, ct);
        pendentes.Clear();

        if (resultado.IsCompleted) break;
    }
}
finally
{
    await reader.CompleteAsync();
}

contexto.RegistrarFimDaLeitura(numeroLinha);
}

private static int ProcessarBloco(ref ReadOnlySequence<byte> buffer, bool
fimDoArquivo,
                                int numeroLinha, List<RegistroRetorno>
destino,
                                IContextoDeLeitura contexto)
{
    var cursor = new SequenceReader<byte>(buffer);

    while (cursor.TryReadTo(out ReadOnlySequence<byte> linha,
(byte)'\n'))

```

```

        ProcessarLinha(linha, ++numeroLinha, destino, contexto);

        if (fimDoArquivo && !cursor.End) // última linha sem quebra
        {
            ProcessarLinha(cursor.UnreadSequence, ++numeroLinha, destino,
contexto);
            cursor.AdvanceToEnd();
        }

        buffer = buffer.Slice(cursor.Position); // o que sobrou é linha
incompleta
        return numeroLinha;
    }

    private static void ProcessarLinha(ReadOnlySequence<byte> sequencia, int
numeroLinha,
                                     List<RegistroRetorno> destino,
IContextoDeLeitura contexto)
    {
        Span<byte> temporario = stackalloc byte[512];
        scoped ReadOnlySpan<byte> linha;

        if (sequencia.IsSingleSegment)
        {
            linha = sequencia.FirstSpan; // caso comum: nenhuma cópia
        }
        else
        {
            if (sequencia.Length > temporario.Length) {
contexto.RegistrarErro(numeroLinha, "linha longa demais"); return; }
            sequencia.CopyTo(temporario); // linha cruzou segmentos: cópia
pequena na pilha
            linha = temporario[..(int)sequencia.Length];
        }

        if (!linha.IsEmpty && linha[^1] == (byte)'\r') linha = linha[..^1];
        if (linha.IsEmpty) return;

        switch (linha[0])
        {
            case (byte)'0': // header: valida banco, tipo de arquivo e data;
layout desconhecido lança LayoutInvalidoException
                return;

            case (byte)'9': // trailer: guarda os totais para a reconciliação
                if (Cnab400Parser.TryParseTrailer(linha, out int quantidade,

```

```

out long total))
        contexto.RegistrarTrailer(quantidade, total);
    else
        contexto.RegistrarErro(numeroLinha, "trailer inválido");
    return;
}

    if (Cnab400Parser.TryParseDetalhe(linha, numeroLinha, out var
registro, out var erro))
        destino.Add(registro);
    else
        contexto.RegistrarErro(numeroLinha, erro!);
}
}

```

Pontos didáticos desse código, um por um:

1. **Separação síncrono/assíncrono.** `Span` não atravessa `await`, então o parsing fica em métodos síncronos que recebem o buffer, e o `await` só aparece para ler mais e para escrever no canal. Esse padrão vale para todo código de alta performance.
2. **ref buffer.** O método síncrono devolve, pelo próprio parâmetro, a parte não consumida. `buffer.Start` passa a ser o início da linha incompleta.
3. **AdvanceTo(buffer.Start, buffer.End).** Consumimos até a última linha completa e dizemos que examinamos tudo. O pipe guarda a linha incompleta e só nos acorda quando chegar mais dado.
4. **scoped.** A palavra-chave diz ao compilador que `linha` não sai do método, o que permite apontá-la tanto para o buffer do pipe quanto para memória `stackalloc`.
5. **Erros não interrompem a leitura.** Linhas inválidas vão para o contexto de leitura com o número da linha, e o trailer tem seus totais guardados ali para a reconciliação. Ao fim, você decide: rejeitar o arquivo inteiro ou aceitar parcialmente. Em retorno bancário, o comum é rejeitar o arquivo se header ou trailer estiverem inválidos e registrar linhas de detalhe inválidas como ocorrências.
6. **Lote local antes do canal.** Os registros de um bloco são acumulados numa lista reutilizada e escritos no canal depois. Isso evita um `await` por linha dentro do parsing.

4.4 Lendo direto do S3, sem disco

O `ResponseStream` de um `GetObjectAsync` é um stream de rede comum. Passe-o direto para o `PipeReader.Create`; nunca copie para um `MemoryStream` primeiro (isso coloca o arquivo inteiro na memória, quase sempre no LOH).

```

var requisicao = new GetObjectRequest { BucketName = bucket, Key = chave };

// Para um chunk do fan-out: só o intervalo de bytes dele
if (chunk is not null)
    requisicao.ByteRange = new ByteRange(chunk.Inicio, chunk.Fim);

using GetObjectResponse resposta = await s3.GetObjectAsync(requisicao, ct);
await using Stream corpo = resposta.ResponseStream;
await leitor.LerAsync(corpo, canal.Writer, contexto, ct);

```

Dois cuidados com streams de rede. Primeiro, uma conexão pode cair no meio: trate `IOException` como erro recuperável e reprocessse o arquivo inteiro (o que só é seguro porque a escrita é idempotente, Módulo 8). Segundo, se o processamento for lento e o canal encher, a leitura fica parada e a conexão HTTP pode expirar. Se isso acontecer com frequência, baixe primeiro para um arquivo temporário local em streaming e processe a partir do disco.

4.5 Integridade: hash calculado enquanto lê

Na ingestão, calcule o SHA-256 no mesmo passo em que grava, sem ler o arquivo duas vezes. `IncrementalHash` aceita blocos:

```

using var hash = IncrementalHash.CreateHash(HashAlgorithmName.SHA256);
byte[] bloco = ArrayPool<byte>.Shared.Rent(81920);
try
{
    int n;
    while ((n = await entrada.ReadAsync(bloco.AsMemory(), ct)) > 0)
    {
        hash.AppendData(bloco, 0, n);
        await destino.WriteAsync(bloco.AsMemory(0, n), ct);
    }
}
finally { ArrayPool<byte>.Shared.Return(bloco); }

string sha256 = Convert.ToHexString(hash.GetHashAndReset());

```

O buffer de 81.920 bytes não é aleatório: fica logo abaixo do limite de 85.000 bytes do LOH, e é o tamanho que o próprio `Stream.CopyToAsync` usa.

4.6 Escrevendo arquivos (remessa ou relatórios)

A escrita segue a mesma lógica ao contrário: formate direto em bytes, num buffer alugado, sem montar strings. Os tipos numéricos têm `TryFormat` para `Span<byte>` (UTF-8):

```
void EscreverValor(Span<byte> destino, long centavos)
{
    // campo numérico de 13 posições com zeros à esquerda
    centavos.TryFormat(destino[..13], out int escritos, "D13");
}
```

Para gravar, use `PipeWriter.Create(stream)` ou um `FileStream` com buffer grande, e escreva sempre num arquivo temporário que é renomeado (ou cujo upload é concluído) só no fim. Quem consome nunca deve enxergar um arquivo pela metade. No S3, use upload multipart com `TransferUtility`, que envia partes em paralelo e só publica o objeto quando todas chegam.

4.7 Arquivos compactados

`GZipStream` funciona em streaming: envolva o stream de origem e passe adiante. Arquivos `.zip` são diferentes: o índice fica no fim do arquivo, então `ZipArchive` precisa de um stream com busca (seek). Com um stream de rede, ele copia tudo para a memória. Para zip vindo do S3, baixe para um arquivo temporário e abra dali. O .NET 10 adicionou APIs assíncronas para `ZipArchive`, o que evita bloquear threads nesse caminho.

Exercício do Módulo 4

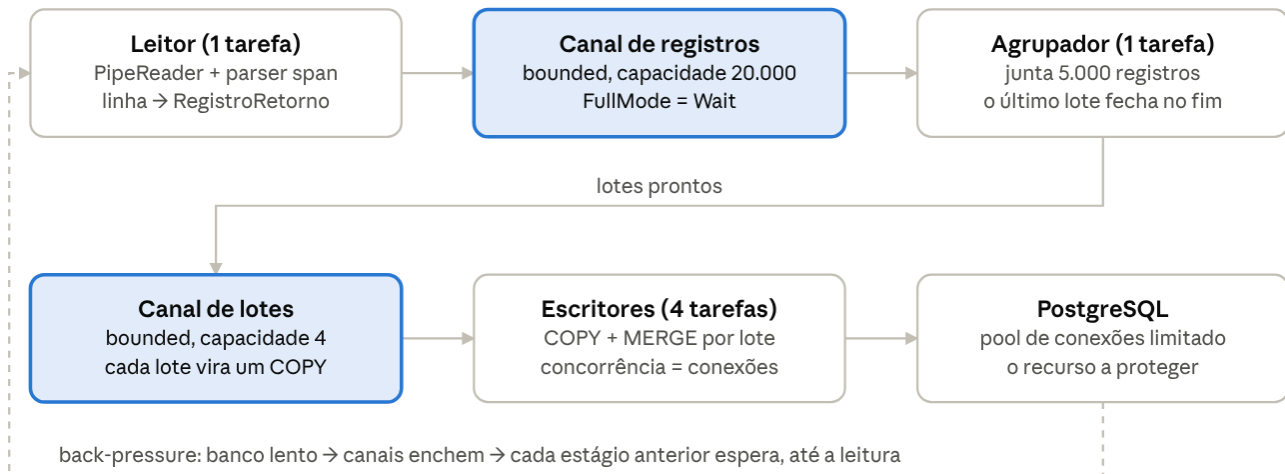
Gere um arquivo sintético de 2 milhões de linhas de 400 posições (cerca de 800 MB). Leia-o com `StreamReader.ReadLineAsync` e depois com o `LeitorCnab400`, usando um canal sem consumidor real (só descarte). Meça com `dotnet-counters monitor` o pico de memória (`working-set`), o número de coletas `gen0/gen2` e o tempo total. Depois force uma linha a cruzar a borda do buffer (ajuste `bufferSize` para um valor que não divida 402) e confirme que o caminho com `stackalloc` funciona.

Módulo 5 — Concorrência e back-pressure

Dentro de cada worker, o processamento é uma linha de montagem: um leitor, um agrupador e alguns escritores, ligados por `Channel<T>` com **capacidade limitada**. O limite é o que importa: quando o banco desacelera, os canais enchem e cada estágio espera o seguinte, de modo que a memória nunca cresce e o banco dita o ritmo.

Canais com limite fazem o banco ditar o ritmo da leitura

Pipeline interno de um worker · em destaque, os canais que transmitem a pressão para trás



pipeline interno do worker · 4 estágios, 2 canais com limite

A seta tracejada é o conceito mais importante do módulo: a pressão caminha para trás sem nenhum código de controle explícito. É o `await WriteAsync` num canal cheio que faz isso.

5.1 Concorrência, paralelismo e o ThreadPool

Concorrência é ter várias tarefas em andamento ao mesmo tempo; **paralelismo** é executá-las literalmente ao mesmo tempo em núcleos diferentes. Trabalho de I/O (esperar o banco, a rede, o disco) precisa de concorrência, não de threads: um `await` libera a thread enquanto espera. Trabalho de CPU (parsing) precisa de paralelismo, limitado ao número de núcleos.

No .NET, o `ThreadPool` executa as continuações dos `await`. Ele cresce devagar quando todas as threads estão ocupadas. Se você **bloquear** threads (`.Result`, `.Wait()`, `Thread.Sleep`, I/O síncrono), o pool fica sem threads livres, as continuações esperam na fila e o sistema inteiro parece travado com CPU baixa. Isso se chama **ThreadPool starvation** e se diagnostica com o contador `threadpool-queue-length` subindo no `dotnet-counters`.

Regra: no worker, tudo que espera é `async` de ponta a ponta. O único trabalho síncrono são os trechos de CPU curtos (parsear um bloco de linhas).

5.2 Channels em detalhe

`System.Threading.Channels` é uma fila produtor-consumidor assíncrona e de alta performance, dentro do processo. Existem dois tipos:

- **Unbounded** (sem limite): `WriteAsync` nunca espera. Se o consumidor for mais lento, a memória cresce até o processo morrer. **Nunca use num pipeline de dados.**
- **Bounded** (com limite): quando cheio, o comportamento é definido por `FullMode`. Com `Wait`, o `WriteAsync` espera abrir espaço. Os outros modos (`DropOldest`, `DropNewest`, `DropWrite`) descartam dados e só servem para telemetria, nunca para registros financeiros.

As opções `SingleWriter` e `SingleReader` não mudam o comportamento, mas permitem ao canal usar uma implementação mais rápida, sem sincronização para o lado único. Declare sempre que for verdade.

O ciclo de vida de um canal tem três regras:

1. Quem escreve chama `Writer.Complete()` quando termina. Sem isso, o leitor espera para sempre.
2. Se quem escreve falhar, chama `Writer.Complete(excecao)`. O leitor recebe a exceção na próxima leitura, e o erro se propaga pela linha de montagem.
3. Para ler com eficiência, use `WaitToReadAsync` + `TryRead` em laço: só há um `await` quando o canal está vazio. `ReadAllAsync` é mais simples e serve para os escritores, que fazem pouco trabalho por item.

5.3 O orquestrador do pipeline

Este é o código que monta a linha de montagem do diagrama para um arquivo. Leia com atenção a parte de erros: se qualquer estágio falhar, todos são cancelados e o erro original sobe.

```
using System Buffers;
using System.ComponentModel.DataAnnotations;
using System.Threading.Channels;
using Microsoft.Extensions.Options;

public sealed record Lote(Guid ArquivoId, int Numero, RegistroRetorno[]
Itens, int Quantidade);

public sealed class ProcessamentoOptions
{
    [Range(1_000, 1_000_000)] public int CapacidadeRegistros { get; set; } =
20_000;
    [Range(100, 50_000)] public int TamanhoLote { get; set; } = 5_000;
    [Range(1, 64)] public int CapacidadeLotes { get; set; } = 4;
    [Range(1, 32)] public int Escritores { get; set; } = 4;
}
```

```

public sealed class PipelineDeArquivo(
    LeitorCnab400 leitor,
    IEscritorDeLotes escritor,
    IOptions<ProcessamentoOptions> opcoes)
{
    public async Task ProcessarAsync(Guid arquivoId, Stream origem,
                                     IContextoDeLeitura contexto,
    CancellationToken ct)
    {
        var o = opcoes.Value;
        using var cts = CancellationTokenSource.CreateLinkedTokenSource(ct);
        CancellationToken token = cts.Token;

        var registros = Channel.CreateBounded<RegistroRetorno>(
            new BoundedChannelOptions(o.CapacidadeRegistros)
            { SingleWriter = true, SingleReader = true, FullMode =
BoundedChannelFullMode.Wait });

        var lotes = Channel.CreateBounded<Lote>(
            new BoundedChannelOptions(o.CapacidadeLotes)
            { SingleWriter = true, SingleReader = false, FullMode =
BoundedChannelFullMode.Wait });

        Task leitura = Estagio(cts, registros.Writer,
                                () => leitor.LerAsync(origem, registros.Writer, contexto,
token));

        Task agrupamento = Estagio(cts, lotes.Writer,
                                    () => AgruparAsync(arquivoId, registros.Reader, lotes.Writer,
o.TamanhoLote, token));

        Task[] escritas = Enumerable.Range(0, o.Escritores)
            .Select(_ => EscreverAsync(lotes.Reader, cts, token))
            .ToArray();

        await Task.WhenAll([leitura, agrupamento, .. escritas]);
    }

    // Executa um estágio; ao terminar, fecha o canal de saída (com ou sem
erro)
    private static async Task Estagio<T>(CancellationTokenSource cts,
ChannelWriter<T> saida,
                                     Func<Task> corpo)
    {
        try
        {

```

```

        await corpo();
        saida.TryComplete();
    }
    catch (Exception ex)
    {
        saida.TryComplete(ex);
        cts.Cancel();
        throw;
    }
}

private static async Task AgruparAsync(Guid arquivoId,
ChannelReader<RegistroRetorno> entrada,
ChannelWriter<Lote> saida, int
tamanho, CancellationToken ct)
{
    var pool = ArrayPool<RegistroRetorno>.Shared;
    RegistroRetorno[] itens = pool.Rent(tamanho); // em todo momento,
'itens' pertence a este método
    int quantidade = 0, numero = 0;

    try
    {
        while (await entrada.WaitToReadAsync(ct))
        {
            while (entrada.TryRead(out RegistroRetorno r))
            {
                itens[quantidade++] = r;
                if (quantidade == tamanho)
                {
                    await saida.WriteAsync(new Lote(arquivoId, ++numero,
itens, quantidade), ct);
                    itens = pool.Rent(tamanho); // o array anterior agora
pertence ao escritor
                    quantidade = 0;
                }
            }
        }

        if (quantidade > 0)
        {
            await saida.WriteAsync(new Lote(arquivoId, ++numero, itens,
quantidade), ct);
            itens = []; // entregue: nada a devolver
        }
    }
}

```

```

        finally
        {
            if (itens.Length > 0) pool.Return(itens); // erro, cancelamento
ou último lote vazio
        }
    }

    private async Task EscreverAsync(ChannelReader<Lote> entrada,
CancellationTokenSource cts,
                                Cancellation token ct)
    {
        try
        {
            await foreach (Lote lote in entrada.ReadAllAsync(ct))
            {
                try { await escritor.GravarAsync(lote, ct); }
                finally {
ArrayPool<RegistroRetorno>.Shared.Return(lote.Itens); }
            }
        }
        catch
        {
            cts.Cancel();
            throw;
        }
    }
}

```

O que observar:

- **Posse de memória explícita.** O array de cada lote é alugado pelo agrupador e devolvido pelo escritor. Depois que o lote entra no canal, o agrupador nunca mais toca nele. Num sistema com pools, deixar claro quem é dono de cada buffer evita os bugs mais difíceis (dados de um lote aparecendo em outro).
- **Cancelamento vinculado.** O `CreateLinkedTokenSource` herda o cancelamento do host (SIGTERM) e permite que um estágio com erro cancele os outros.
- **Task.WhenAll com spread.** A expressão de coleção `[leitura, agrupamento, .. escritas]` junta tudo. Se algo falhar, `WhenAll` lança a primeira exceção depois que todas as tarefas terminarem, então nenhuma fica órfã.
- **Nenhum Task.Run.** Os estágios são I/O-bound e o parsing é rápido o bastante para rodar nas continuações. Se o parsing virasse gargalo de CPU, aí sim valeria ter dois ou mais leitores em chunks diferentes.

5.4 Vários arquivos ao mesmo tempo

O pipeline acima processa um arquivo. O worker consome a fila e pode processar alguns arquivos em paralelo. Use `Parallel.ForEachAsync` com limite explícito, ou um `SemaphoreSlim`:

```
await Parallel.ForEachAsync(
    ConsumirMensagensAsync(ct), // IEnumerable<MensagemArquivo>
    new ParallelOptions { MaxDegreeOfParallelism = 2, CancellationToken = ct
},
    async (mensagem, token) => await ProcessarMensagemAsync(mensagem,
        token));
```

Cuidado com a multiplicação: 2 arquivos em paralelo × 4 escritores = 8 conexões simultâneas por réplica. Com 6 réplicas, são 48 conexões. É esse número total que precisa caber no banco.

5.5 Dimensionando a concorrência

A concorrência certa não é "a maior possível". A partir de um ponto, mais escritores só aumentam a disputa por locks, I/O de disco e CPU do banco, e a vazão **cai**. O processo para encontrar o ponto é empírico:

1. Comece com a Lei de Little (Módulo 1) para ter uma estimativa.
2. Rode um teste de carga com 1, 2, 4, 8 e 16 escritores totais, medindo linhas por segundo e latência por lote.
3. Escolha o menor número que entrega cerca de 90% da vazão máxima. Ele deixa folga para outros sistemas que usam o mesmo banco.

No Npgsql, configure o pool na connection string: `Maximum Pool Size` deve ser pelo menos o número de escritores da réplica mais uma margem. Se muitas réplicas compartilham o banco, um pooler como **PgBouncer** em modo transação reduz o número de conexões reais (com a ressalva de que o `COPY` e prepared statements exigem cuidado nesse modo).

5.6 Quando a ordem importa

Com vários escritores, lotes são gravados fora de ordem. Isso não é problema se cada linha é independente. Mas se duas linhas sobre o **mesmo** título precisam ser aplicadas em ordem (uma entrada e depois uma liquidação), você tem duas opções:

- **Particionar por chave:** em vez de um canal de lotes, crie N canais e mande cada registro para o canal `hash(chave) % N`. Cada canal tem um único escritor. Todos os eventos de uma chave passam pelo mesmo escritor, em ordem.

- **Resolver no banco:** grave tudo em staging com o número da linha, e faça a consolidação em SQL ordenando por chave e linha (`ORDER BY` ou funções de janela). Essa costuma ser a opção mais simples e robusta para arquivos.

5.7 Armadilhas clássicas

- `async void` fora de handlers de evento: exceções derrubam o processo sem passar pelo seu `catch`.
- Fire-and-forget (`_ = FazerAlgoAsync()`): ninguém observa a falha nem espera o término no shutdown.
- `lock` com `await` dentro: não compila, e a tentação de contornar com `Monitor` é pior. Use `SemaphoreSlim.WaitAsync` ou, melhor, desenhe sem estado compartilhado (canais).
- `Task.Run` dentro de laço para "paralelizar" I/O: cria concorrência sem limite. Sempre limite explicitamente.
- Capturar `DbContext` em várias tarefas: `DbContext` não é thread-safe. Cada escritor precisa do seu (`IDbContextFactory<T>`) ou da sua conexão.

Exercício do Módulo 5

Monte o `PipelineDeArquivo` com um `IEscritorDeLotes` falso que faz `await Task.Delay(50)` por lote. Rode o arquivo do Módulo 4 e observe no `dotnet-counters` que a memória fica estável mesmo com o escritor lento. Depois troque o canal de registros para unbounded e veja a memória crescer. Por fim, faça o escritor lançar exceção no lote 10 e confirme que o leitor para logo em seguida e que a exceção original chega ao chamador.

Módulo 6 — Escrita em massa no banco

A forma mais rápida e segura de gravar milhões de linhas no PostgreSQL é: para cada lote, abrir uma transação, carregar o lote numa tabela temporária com `COPY` binário e consolidar na tabela final com um único `INSERT ... SELECT ... ON CONFLICT`, gravando os eventos da outbox no mesmo comando. Uma ida ao banco para os dados, uma para a consolidação, um commit.

6.1 Por que linha a linha é tão lento

Inserir uma linha por vez paga quatro custos repetidos, e nenhum deles é o custo de gravar o dado em si:

1. **Ida e volta de rede:** cada comando espera a resposta antes do próximo.

2. **Parse e planejamento:** o banco analisa o SQL a cada vez (prepared statements reduzem, mas não zeram).
3. **Commit:** em modo autocommit, cada linha é uma transação, e cada commit espera o WAL ser gravado em disco (fsync).
4. **Manutenção de índices linha a linha:** cada índice da tabela é atualizado individualmente.

As técnicas de escrita em massa atacam esses custos em graus diferentes:

Técnica	Idas ao banco por lote de 5.000	Prós	Contras
<code>SaveChanges</code> por linha	5.000 + 5.000 commits	Simples	Inaceitável em volume
<code>AddRange</code> + um <code>SaveChanges</code>	Algumas dezenas (EF agrupa comandos)	Usa o modelo do EF	Change tracker caro; ainda um <code>INSERT</code> por linha no servidor
<code>INSERT ... VALUES (...), (...)</code>	1 a poucas	SQL comum	Limite de 65.535 parâmetros por comando; SQL gigante
<code>INSERT ... SELECT FROM unnest(@arrays)</code>	1	Um parâmetro por coluna; funciona com PgBouncer	Montar arrays por coluna
<code>COPY ... FROM STDIN (FORMAT BINARY)</code>	1 (stream)	O mais rápido; protocolo feito para carga	Não faz upsert sozinho; precisa de staging

A combinação vencedora é a última linha mais uma consolidação em SQL: `COPY` resolve a velocidade de transferência, e o `INSERT ... SELECT` resolve idempotência e regras de negócio, tudo de forma set-based (o banco processa o conjunto inteiro de uma vez, com um plano só).

6.2 O modelo de dados

A tabela final tem uma chave natural que torna a gravação idempotente: o par (arquivo, número da linha) identifica um registro de forma única e estável. Reprocessar o arquivo produz exatamente os mesmos pares.

```

CREATE TABLE retorno_detalhe (
    id                bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    arquivo_id        uuid      NOT NULL REFERENCES arquivo (id),
    numero_linha       int       NOT NULL,
    nosso_numero       bigint    NOT NULL,
    codigo_ocorrencia  smallint  NOT NULL,
    data_ocorrencia    date      NOT NULL,
    valor_pago_centavos bigint    NOT NULL,
    gravado_em         timestampz NOT NULL DEFAULT now(),
    CONSTRAINT uq_retorno_linha UNIQUE (arquivo_id, numero_linha)
);

CREATE INDEX ix_retorno_nosso_numero ON retorno_detalhe (nosso_numero);

CREATE TABLE outbox (
    id                uuid PRIMARY KEY,
    tipo              text      NOT NULL,
    chave             text      NOT NULL,
    payload           jsonb      NOT NULL,
    criado_em         timestampz NOT NULL DEFAULT now(),
    publicado_em      timestampz NULL
);

CREATE INDEX ix_outbox_pendentes ON outbox (criado_em) WHERE publicado_em IS NULL;

```

Repare em três escolhas. Valores em centavos como `bigint`, coerente com o `long` do C#. Um índice parcial na outbox, que só contém eventos pendentes e por isso fica sempre pequeno. E poucos índices na tabela final: cada índice a mais custa em toda inserção, então só crie os que as consultas reais exigem.

O mesmo modelo no EF Core fica num projeto `Cobranca.Dados`, compartilhado entre os workers deste curso e a API do curso de Minimal APIs. As migrations do EF passam a ser a fonte da verdade do schema; o SQL acima é o que elas devem gerar.

```

public sealed class RetornoDetalhe
{
    public long Id { get; init; }
    public Guid ArquivoId { get; init; }
    public int NumeroLinha { get; init; }
    public long NossoNumero { get; init; }
    public short CodigoOcorrencia { get; init; }
    public DateOnly DataOcorrencia { get; init; }
    public long ValorPagoCentavos { get; init; }
    public DateTimeOffset GravadoEm { get; init; }
}

```

```

}

public sealed class CobrancaDb(DbContextOptions<CobrancaDb> opcoes) :
DbContext(opcoes)
{
    public DbSet<Arquivo> Arquivos => Set<Arquivo>();
    public DbSet<RetornoDetalhe> RetornoDetalhes => Set<RetornoDetalhe>();
    public DbSet<MensagemOutbox> Outbox => Set<MensagemOutbox>();

    protected override void OnModelCreating(ModelBuilder model)
    {
        model.Entity<RetornoDetalhe>(e =>
        {
            e.ToTable("retorno_detalhe");
            e.HasIndex(d => new { d.ArquivoId, d.NumeroLinha }).IsUnique();
            e.HasIndex(d => d.NossoNumero);
        });

        model.Entity<MensagemOutbox>(e =>
        {
            e.ToTable("outbox");
            e.Property(m => m.Payload).HasColumnType("jsonb");
            e.HasIndex(m => m.CriadoEm).HasFilter("publicado_em IS NULL"); //
o índice parcial
        });
    }
}

// Registro: nomes em snake_case (pacote EFCore.NamingConventions)
builder.Services.AddDbContextFactory<CobrancaDb>((sp, o) => o
    .UseNpgsql(sp.GetRequiredService<NpgsqlDataSource>())
    .UseSnakeCaseNamingConvention());

```

No worker, registre o contexto com `AddDbContextFactory` em vez de `AddDbContext`. O worker é um singleton que vive o processo inteiro; com a fábrica, cada operação cria e descarta o seu próprio `DbContext`, sem dependência cativa (o curso de Minimal APIs explica o termo no Módulo 2). O escritor de lotes nem precisa do EF: usa o `NpgsqlDataSource`, que é thread-safe e também singleton.

6.3 COPY binário com Npgsql

O Npgsql expõe o protocolo `COPY` por `BeginBinaryImportAsync`. Você escreve linha por linha, coluna por coluna, e o driver serializa tudo num stream contínuo. Três regras:

- A ordem e os tipos das colunas no `WriteAsync` devem bater exatamente com a lista do comando `COPY`. Informe o `NpgsqlDbType` para evitar inferência errada.

- **Chame `CompleteAsync()`**. Se o importador for descartado sem `Complete`, o `COPY` inteiro é cancelado. Esse comportamento é ótimo para atomicidade, mas é um bug clássico quando esquecido.
- Use `NpgsqlDataSource` (registrado com `AddNpgsqlDataSource` no DI) para obter conexões. Ele gerencia o pool e é a API recomendada nas versões atuais do Npgsql.

6.4 O escritor completo: uma transação por lote

```
using Npgsql;
using NpgsqlTypes;

public interface IEscritorDeLotes
{
    Task GravarAsync(Lote lote, CancellationToken ct);
}

public sealed class EscritorPostgres(NpgsqlDataSource dataSource) :
    IEscritorDeLotes
{
    private const string CriarTemporaria = """
        CREATE TEMP TABLE tmp_retorno (
            arquivo_id uuid, numero_linha int, nosso_numero bigint,
            codigo_ocorrencia smallint, data_ocorrencia date,
            valor_pago_centavos bigint
        ) ON COMMIT DROP;
        """;

    private const string Copy = """
        COPY tmp_retorno (arquivo_id, numero_linha, nosso_numero,
            codigo_ocorrencia, data_ocorrencia,
            valor_pago_centavos)
        FROM STDIN (FORMAT BINARY)
        """;

    private const string Consolidar = """
        WITH inseridos AS (
            INSERT INTO retorno_detalhe AS d
                (arquivo_id, numero_linha, nosso_numero,
                 codigo_ocorrencia, data_ocorrencia, valor_pago_centavos)
            SELECT arquivo_id, numero_linha, nosso_numero,
                codigo_ocorrencia, data_ocorrencia, valor_pago_centavos
            FROM tmp_retorno
            ON CONFLICT (arquivo_id, numero_linha) DO NOTHING
            RETURNING d.id, d.nosso_numero, d.codigo_ocorrencia,
            d.valor_pago_centavos
```

```

    )
    INSERT INTO outbox (id, tipo, chave, payload)
    SELECT gen_random_uuid(), 'retorno.ocorrencia', i.nosso_numero::text,
           jsonb_build_object('detalheId', i.id, 'nossoNumero',
                             i.nosso_numero,
                             'ocorrencia', i.codigo_ocorrencia,
                             'valorPagoCentavos', i.valor_pago_centavos)
    FROM inseridos i;
    """;

    public async Task GravarAsync(Lote lote, CancellationToken ct)
    {
        await using NpgsqlConnection conexao = await
dataSource.OpenConnectionAsync(ct);
        await using NpgsqlTransaction tx = await
conexao.BeginTransactionAsync(ct);

        await using (var criar = new NpgsqlCommand(CriarTemporaria, conexao,
tx))
            await criar.ExecuteNonQueryAsync(ct);

        await using (NpgsqlBinaryImporter copy = await
conexao.BeginBinaryImportAsync(Copy, ct))
        {
            for (int i = 0; i < lote.Quantidade; i++)
            {
                RegistroRetorno r = lote.Itens[i];
                await copy.StartRowAsync(ct);
                await copy.WriteAsync(lote.ArquivoId, NpgsqlDbType.Uuid, ct);
                await copy.WriteAsync(r.NumeroLinha, NpgsqlDbType.Integer,
ct);
                await copy.WriteAsync(r.NossoNumero, NpgsqlDbType.Bigint,
ct);
                await copy.WriteAsync((short)r.CodigoOcorrencia,
NpgsqlDbType.Smallint, ct);
                await copy.WriteAsync(r.DataOcorrencia, NpgsqlDbType.Date,
ct);
                await copy.WriteAsync(r.ValorPagoCentavos,
NpgsqlDbType.Bigint, ct);
            }
            await copy.CompleteAsync(ct);
        }

        await using (var consolidar = new NpgsqlCommand(Consolidar, conexao,
tx))
            await consolidar.ExecuteNonQueryAsync(ct);
    }

```

```

        await tx.CommitAsync(ct);
    }
}

```

Por que esse desenho é tão bom, ponto a ponto:

- **Atomicidade por lote.** Dados, consolidação e outbox estão na mesma transação. Ou tudo do lote entra, ou nada entra.
- **Idempotência automática.** Se o lote for regravado (retry, reprocesso do arquivo), o `ON CONFLICT DO NOTHING` ignora as linhas existentes, e o `RETURNING` só devolve as linhas realmente inseridas. Portanto a outbox **não recebe eventos duplicados**. Esse é o detalhe mais elegante do módulo.
- **Tabela temporária por transação.** `ON COMMIT DROP` faz a tabela desaparecer sozinha. Tabelas temporárias não geram WAL nem disputam com outras sessões, então o `COPY` nelas é muito rápido.
- **Uma consolidação set-based.** O banco faz um único plano para 5.000 linhas, usando o índice único para detectar conflitos.

Um refinamento para volume muito alto: criar e destruir uma tabela temporária a cada lote gera escrita nos catálogos do sistema (`pg_class` , `pg_attribute`) e, com milhares de lotes por minuto, esses catálogos incham. A alternativa é criar a tabela uma vez por conexão física com `CREATE TEMP TABLE IF NOT EXISTS ... ON COMMIT DELETE ROWS` : como o pool reaproveita conexões, a tabela sobrevive entre lotes e só é esvaziada a cada commit.

Um detalhe de desempenho: o `WriteAsync` por coluna parece caro, mas o Npgsql escreve num buffer interno e só vai à rede quando ele enche, então a maioria das chamadas completa de forma síncrona.

6.5 ON CONFLICT x MERGE

`INSERT ... ON CONFLICT` é a ferramenta padrão de upsert no PostgreSQL. `DO NOTHING` serve para dados imutáveis (eventos do arquivo). `DO UPDATE SET ... = EXCLUDED...` serve para estado que muda (situação atual de um título), e você pode condicionar a atualização: `WHERE d.atualizado_em < EXCLUDED.atualizado_em` , para que um evento antigo nunca sobrescreva um novo.

Uma restrição importante: um mesmo comando `ON CONFLICT DO UPDATE` não pode afetar a mesma linha duas vezes. Se o lote tiver duas linhas para o mesmo título, o comando falha. Resolva deduplicando na tabela temporária com `SELECT DISTINCT ON (nosso_numero) ... ORDER BY nosso_numero, numero_linha DESC` .

MERGE (PostgreSQL 15+) é o comando do padrão SQL, com cláusulas **WHEN MATCHED** e **WHEN NOT MATCHED**. Ele é mais expressivo (permite atualizar, inserir e apagar no mesmo comando, com condições diferentes) e, desde o PostgreSQL 17, aceita **RETURNING**. Porém, sob concorrência, **MERGE** não tem a garantia atômica de "inserir ou atualizar" do **ON CONFLICT**: dois **MERGE** simultâneos podem tentar inserir a mesma chave e um falhará por violação de unicidade. Regra prática: use **ON CONFLICT** quando escritores concorrentes podem tocar as mesmas chaves; use **MERGE** quando a lógica for rica e as chaves de cada transação não se sobrepuserem.

6.6 Alternativa sem COPY: unnest

Quando **COPY** não é viável (PgBouncer em modo transação com restrições, ou lotes pequenos), passe arrays por coluna e expanda no servidor:

```
INSERT INTO retorno_detalhe (arquivo_id, numero_linha, nosso_numero,
                             codigo_ocorrendia,
                             data_ocorrendia, valor_pago_centavos)
SELECT @arquivo_id, *
FROM unnest(@linhas::int[], @nossos_numeros::bigint[],
            @ocorrencias::smallint[],
            @datas::date[], @valores::bigint[])
ON CONFLICT (arquivo_id, numero_linha) DO NOTHING;
```

No C#, cada parâmetro recebe um array (`int[]`, `long[]`, `DateOnly[]`). São seis parâmetros, seja o lote de 10 ou de 10.000 linhas, e o plano pode ser reaproveitado.

6.7 E o EF Core 10?

O EF Core continua sendo a melhor escolha para o modelo de dados, migrations e operações sobre poucas entidades (atualizar o status do arquivo, criar chunks). Para o caminho de alto volume, use-o com critério:

- **ExecuteUpdateAsync** e **ExecuteDeleteAsync** geram um único **UPDATE / DELETE** set-based, sem carregar entidades. No EF Core 10, **ExecuteUpdateAsync** aceita um lambda comum (não apenas expressão), o que permite montar os setters condicionalmente.
- **AddRange** + **SaveChangesAsync** funciona para lotes de centenas de linhas, com `ChangeTracker.AutoDetectChangesEnabled = false` e um `DbContext` novo por lote (o change tracker cresce e fica lento). Para milhares, prefira **COPY**.
- **Consultas SQL cruas** (`Database.SqlQuery<T>` e `ExecuteSqlAsync`) deixam você usar SQL específico do PostgreSQL mantendo o EF para o resto.
- Bibliotecas de bulk para EF (como `EFCore.BulkExtensions`) existem; avalie a licença antes de adotar em ambiente corporativo.

Um padrão saudável é conviver: o `DbContext` e o `NpgsqlDataSource` usam a mesma connection string, o EF cuida do modelo e o escritor de lotes usa Npgsql direto.

6.8 Ajustes do lado do banco

- **Tamanho do lote:** entre 1.000 e 10.000 linhas costuma ser o ponto ótimo. Lotes maiores seguram locks e memória por mais tempo e tornam o retry mais caro. Meça.
- **Transações curtas:** transações longas impedem o autovacuum de limpar versões antigas de linhas (bloat) em **todas** as tabelas. Uma transação por lote, nunca uma por arquivo de milhões de linhas.
- **synchronous_commit**: com `SET LOCAL synchronous_commit = off` na transação, o commit não espera o fsync. Um crash pode perder os últimos milissegundos de commits, mas nunca corrompe o banco. É aceitável **apenas** se o dado pode ser reconstruído a partir do arquivo original e a reconciliação detecta a lacuna. Para dados financeiros, avalie com o time de dados antes.
- **Autovacuum:** tabelas que só recebem inserções também precisam de vacuum (para o visibility map e para evitar o wraparound de IDs de transação). Em tabelas de alto volume, ajuste `autovacuum_vacuum_insert_scale_factor` por tabela.
- **Carga inicial ou reprocesso massivo:** para carregar centenas de milhões de linhas de uma vez, é mais rápido remover índices secundários, carregar e recriá-los depois. Nunca faça isso com o sistema em produção recebendo escrita.

6.9 Notas para SQL Server

Se o banco for SQL Server, a estrutura é a mesma e as ferramentas mudam: `SqlBulkCopy` faz o papel do `COPY` (passe um `IDataReader` para transmitir sem materializar o lote, e use `SqlBulkCopyOptions.TableLock` para tabelas de staging), tabelas `#temporarias` fazem o papel da `TEMP TABLE`, e a consolidação usa `MERGE` com `HOLDLOCK` ou `INSERT ... WHERE NOT EXISTS`. Table-valued parameters substituem o `unnest`.

Exercício do Módulo 6

Crie as tabelas acima num PostgreSQL local (Docker). Implemente três escritores, `SaveChanges` em lote, `unnest` e `COPY` + consolidação, e grave 1 milhão de registros com cada um, em lotes de 5.000 e com 4 escritores concorrentes. Anote linhas por segundo. Depois rode o `COPY` de novo com o mesmo arquivo e confirme duas coisas: a contagem de `retorno_detalhe` não mudou, e nenhuma linha nova apareceu na `outbox`.

Módulo 7 — Leitura em massa

Ler tabelas grandes bem significa quatro coisas: trazer só as colunas necessárias, consumir as linhas em streaming em vez de materializar listas, paginar por chave (keyset) em vez de `OFFSET`, e ter um índice que transforme a consulta numa varredura curta. Particionamento entra quando a tabela passa de centenas de milhões de linhas.

7.1 Streaming de resultados

`ToListAsync()` carrega o resultado inteiro na memória antes de você ver a primeira linha. Para resultados grandes, use `AsAsyncEnumerable()`, que entrega cada linha assim que o driver a lê da rede:

```
await using var db = await fabrica.CreateDbContextAsync(ct);

IAsyncEnumerable<DetalheParaConciliar> linhas = db.RetornoDetalhes
    .AsNoTracking()
    .Where(d => d.DataOcorrencia == dia)
    .Select(d => new DetalheParaConciliar(d.Id, d.NossoNumero,
d.ValorPagoCentavos))
    .AsAsyncEnumerable();

await foreach (DetalheParaConciliar linha in linhas.WithCancellation(ct))
{
    // processa uma por vez; memória constante
}
```

Três detalhes fazem a diferença. `AsNoTracking()` desliga o change tracker, que guardaria uma cópia de cada entidade. A projeção com `Select` para um `record` traz só três colunas em vez da linha inteira, o que pode permitir que o banco responda só com o índice. E a fonte é um `IDbContextFactory<T>`, para que cada operação de longa duração tenha seu próprio contexto.

Cuidado: enquanto você itera, a conexão e uma transação implícita ficam abertas. Uma leitura em streaming que demora 40 minutos segura um snapshot do MVCC por 40 minutos, o que impede o autovacuum de limpar versões antigas de linhas no banco inteiro. Para jobs longos, prefira ler em páginas (próxima seção), cada uma numa consulta curta.

7.2 Por que OFFSET não escala

`OFFSET 10000000 LIMIT 1000` obriga o banco a ler e descartar um milhão de linhas antes de devolver mil. Cada página fica mais lenta que a anterior, e se houver inserções durante a leitura, linhas aparecem duplicadas ou somem entre páginas.

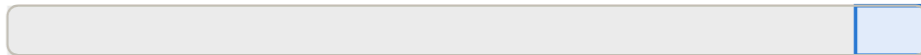
Keyset pagination (ou seek method) usa a última chave vista como ponto de partida:

```
SELECT id, nosso_numero, valor_pago_centavos
FROM retorno_detalhe
WHERE data_ocorrendia = @dia
      AND id > @ultimo_id
ORDER BY id
LIMIT 5000;
```

OFFSET lê e descarta tudo antes da página; keyset desce direto até ela

Página 1.001 de uma tabela, com 1.000 linhas por página · fora de escala

OFFSET 1000000
LIMIT 1000



o custo cresce a cada página

uma descida no índice até a última chave vista

WHERE id > @ultimo_id
LIMIT 1000



custo constante em qualquer página

☒ a página devolvida

☐ lido e descartado

OFFSET x keyset · a mesma página, dois custos

Com **OFFSET**, a página 1.001 obriga o banco a percorrer um milhão de linhas antes de devolver mil; com **keyset**, ele desce no índice direto até a última chave vista e lê só as mil.

Com um índice em **(data_ocorrendia, id)**, cada página é uma descida na árvore B mais uma leitura sequencial de 5.000 entradas. A milionésima página custa o mesmo que a primeira. No EF Core:

```
long ultimoId = 0;
while (true)
{
    var pagina = await db.RetornoDetalhes.AsNoTracking()
        .Where(d => d.DataOcorrendia == dia && d.Id > ultimoId)
        .OrderBy(d => d.Id)
        .Select(d => new DetalheParaConciliar(d.Id, d.NossoNumero,
        d.ValorPagoCentavos))
        .Take(5000)
        .ToListAsync(ct);

    if (pagina.Count == 0) break;
    await ProcessarPaginaAsync(pagina, ct);
    ultimoId = pagina[^1].Id;
}
```

Quando a ordenação é por mais de uma coluna (por exemplo, data e depois id), use a comparação de tuplas do PostgreSQL, que o índice composto atende diretamente: `WHERE (data_ocorrencia, id) > (@data, @id) ORDER BY data_ocorrencia, id`. Esse é um bom caso para SQL cru via `SqlQuery<T>`.

Keyset também resolve a paralelização de leituras: divida o intervalo de ids em faixas (`id BETWEEN a AND b`) e dê uma faixa para cada worker.

7.3 Exportações: COPY TO

Para exportar milhões de linhas (para um arquivo, para outro sistema), o caminho mais rápido é o mesmo protocolo da escrita, ao contrário:

```
await using var conexao = await dataSource.OpenConnectionAsync(ct);
await using NpgsqlBinaryExporter export = await
conexao.BeginBinaryExportAsync(
    "COPY (SELECT nosso_numero, valor_pago_centavos FROM retorno_detalhe
WHERE data_ocorrencia = '2026-10-05') TO STDOUT (FORMAT BINARY)", ct);

while (await export.StartRowAsync(ct) != -1)
{
    long nossoNumero = await export.ReadAsync<long>(ct);
    long valor = await export.ReadAsync<long>(ct);
    // escreve na saída (arquivo, pipe, upload multipart)
}
```

O comando `COPY` não aceita parâmetros; se a consulta tiver valores variáveis, valide e formate-os com muito cuidado (datas e números tipados, nunca texto vindo de fora), ou crie uma view ou função e chame-a.

7.4 EF Core x Dapper x Npgsql puro

Ferramenta	Melhor para	Custo por linha	Observações
EF Core (tracking)	Agregados de domínio, poucas linhas, regras de negócio	Alto	Change tracker e identity resolution
EF Core (<code>AsNoTracking</code> + projeção)	Consultas de leitura tipadas com LINQ	Baixo	Mantém migrations e modelo único

Ferramenta	Melhor para	Custo por linha	Observações
Dapper	SQL escrito à mão, mapeamento simples	Muito baixo	Ótimo com SQL específico do PostgreSQL
Npgsql puro (<code>DbDataReader</code> , <code>COPY</code>)	Hot path, exportações, carga	Mínimo	Mais código, controle total

Não há vencedor único. Em sistemas de alto volume, os três convivem: EF para o modelo e as telas, Dapper ou SQL cru para relatórios, Npgsql puro para carga e exportação. Para consultas EF executadas milhões de vezes (como buscar o status de um arquivo), `EF.CompileAsyncQuery` remove o custo de tradução do LINQ a cada chamada.

7.5 Índices: o que realmente importa

Um índice B-tree composto é uma lista ordenada pela primeira coluna, depois pela segunda, e assim por diante. Disso saem as regras:

- **Colunas de igualdade primeiro, colunas de intervalo ou ordenação depois.** Para `WHERE cedente_id = @c AND data_ocorrendia BETWEEN @a AND @b`, o índice certo é `(cedente_id, data_ocorrendia)`, não o contrário.
- **INCLUDE para cobrir a consulta.** `CREATE INDEX ... ON retorno_detalhe (data_ocorrendia, id) INCLUDE (nosso_numero, valor_pago_centavos)` permite um *index-only scan*: o banco responde sem tocar a tabela.
- **Índices parciais** para subconjuntos pequenos e muito consultados, como a outbox pendente do Módulo 6.
- **BRIN** para colunas que crescem junto com a ordem física da tabela (como `gravado_em` numa tabela só de inserção). Ele guarda o mínimo e o máximo de cada bloco de páginas e ocupa uma fração mínima do espaço de um B-tree.
- **Cada índice tem um preço na escrita.** Antes de criar, pergunte qual consulta ele atende e com que frequência.

Para saber se o índice está sendo usado, leia o plano real: `EXPLAIN (ANALYZE, BUFFERS) SELECT ...`. Procure `Index Scan` ou `Index Only Scan` em vez de `Seq Scan` em tabelas grandes, compare `rows` estimado com o real (estimativas muito erradas indicam estatísticas desatualizadas: rode `ANALYZE`), e olhe `shared read` (páginas lidas do disco) versus `shared hit` (páginas do cache).

7.6 Particionamento: quando a tabela fica grande demais

Pela conta do Módulo 1, a tabela de detalhes cresce dezenas de GB por dia.

Particionamento declarativo por intervalo de data divide a tabela lógica em tabelas físicas menores:

```
CREATE TABLE retorno_detalhe (
  id          bigint GENERATED ALWAYS AS IDENTITY,
  data_ocorrencia date NOT NULL,
  arquivo_id  uuid NOT NULL,
  numero_linha int NOT NULL,
  nosso_numero bigint NOT NULL,
  valor_pago_centavos bigint NOT NULL,
  PRIMARY KEY (id, data_ocorrencia),
  UNIQUE (arquivo_id, numero_linha, data_ocorrencia)
) PARTITION BY RANGE (data_ocorrencia);

CREATE TABLE retorno_detalhe_2026_10 PARTITION OF retorno_detalhe
FOR VALUES FROM ('2026-10-01') TO ('2026-11-01');
```

O que você ganha:

- **Partition pruning:** uma consulta com `WHERE data_ocorrencia = ...` só lê a partição do mês. Índices de cada partição são menores e cabem melhor em memória.
- **Retenção instantânea:** apagar um mês antigo é `DETACH PARTITION + DROP TABLE`, sem gerar bilhões de linhas mortas como um `DELETE` geraria.
- **Manutenção por partição:** vacuum e reindex atuam em pedaços menores.

O preço: toda restrição `UNIQUE` e a chave primária precisam incluir a coluna de partição. Isso afeta a idempotência: a chave natural passa a ser (arquivo, linha, data), e a data precisa ser determinística a partir do arquivo (por exemplo, a data de ocorrência da própria linha). Consultas sem a coluna de partição no filtro percorrem todas as partições. Crie partições futuras com antecedência, com um job agendado ou a extensão `pg_partman`.

7.7 Separando leitura de escrita

Quando relatórios e conciliações pesadas começam a competir com a carga, mova as leituras analíticas para uma **réplica de leitura**. Tenha em mente o atraso de replicação: um dado gravado agora pode levar alguns instantes para aparecer na réplica. Fluxos que precisam ler o que acabaram de escrever (como o fan-in que verifica se todos os chunks terminaram) devem ler do primário.

No MVCC do PostgreSQL, leitores não bloqueiam escritores e vice-versa. O risco real das leituras pesadas no primário não é lock, é I/O, CPU e snapshots longos. Configure `statement_timeout` para consultas ad hoc e `idle_in_transaction_session_timeout` para conexões esquecidas abertas.

Exercício do Módulo 7

Com a tabela do Módulo 6 carregada com alguns milhões de linhas, compare a página 1 e a página 500 usando `OFFSET` e usando keyset, com `EXPLAIN (ANALYZE, BUFFERS)`. Depois crie o índice com `INCLUDE` e confirme o `Index Only Scan` (rode `VACUUM` antes, porque ele depende do visibility map). Por fim, recrie a tabela particionada por mês, carregue dois meses e verifique no plano que uma consulta de um dia toca só uma partição.

Módulo 8 — Confiabilidade: quando algo quebra no meio

Um pipeline confiável não é um pipeline que não falha; é um em que **qualquer passo pode ser repetido com segurança**. A receita é a equação do Módulo 1: entrega pelo menos uma vez (filas, retries, leases) mais idempotência em cada camada (chaves naturais, `ON CONFLICT`, outbox) dá o efeito de exatamente uma vez.

8.1 O mapa de falhas

Antes de escolher técnicas, liste o que pode dar errado. Cada linha desta tabela precisa ter uma resposta no seu design:

Falha	Exemplo	Resposta do design
Mesmo arquivo enviado duas vezes	Banco reenvia o retorno do dia	<code>UNIQUE</code> no hash na ingestão
Worker morre no meio	OOM, deploy, nó do cluster removido	Fila reentrega, lease expira, escrita idempotente
Erro transitório	Timeout de rede, failover do banco, deadlock	Retry com backoff e jitter
Erro permanente	Layout inválido, campo obrigatório vazio	Sem retry: <code>REJEITADO</code> ou DLQ
Gravou no banco, não publicou o evento	Broker fora do ar após o commit	Outbox transacional
Evento publicado duas vezes	Relay cai depois de publicar e antes de marcar	Consumidor deduplica por ID do evento

Falha	Exemplo	Resposta do design
Dado incompleto silencioso	Bug perde linhas sem erro	Reconciliação com o trailer do arquivo

8.2 Idempotência camada por camada

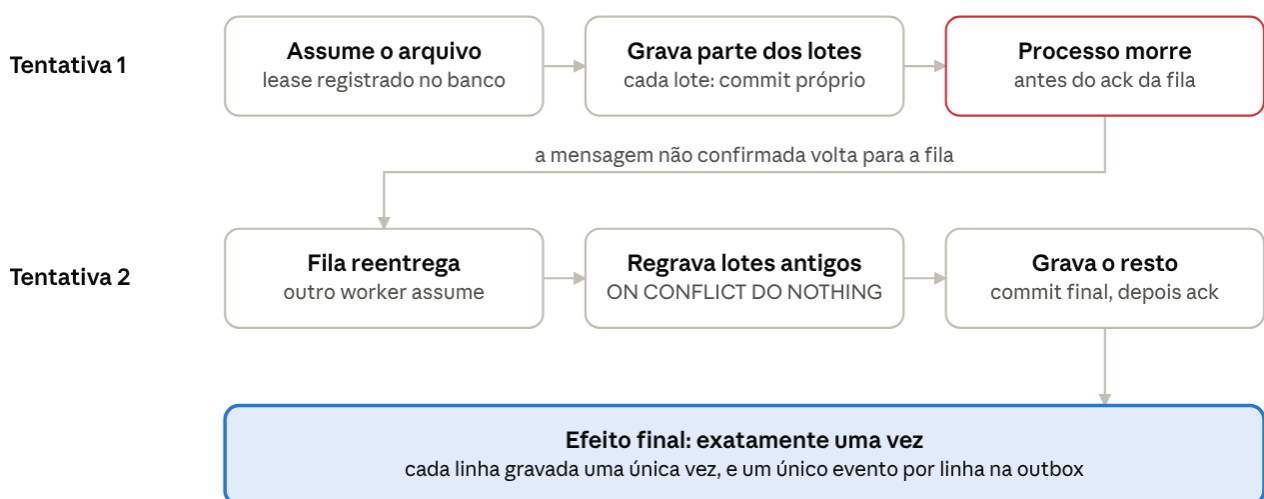
Idempotência não é uma técnica, é uma propriedade que cada passo precisa ter. Revise o pipeline inteiro sob essa lente:

1. **Ingestão:** o hash SHA-256 do conteúdo é único por cliente na tabela `arquivo`. Reenvio vira no-op.
2. **Tomada do arquivo:** um `UPDATE` condicional garante que só um worker por vez processa cada arquivo (próxima seção).
3. **Escrita:** a chave (arquivo, linha) com `ON CONFLICT DO NOTHING` torna regravações inofensivas (Módulo 6).
4. **Eventos:** a outbox só recebe eventos de linhas realmente inseridas, então reprocessar não gera eventos novos.
5. **Consumidores:** cada evento tem um ID estável; quem consome registra os IDs já processados numa tabela `inbox` e ignora repetições.

O teste de fogo: "posso reprocessar este arquivo do zero agora, em produção, sem medo?". Se a resposta for sim, o sistema é idempotente.

Pelo menos uma vez + idempotência = efeito de exatamente uma vez

Um arquivo processado duas vezes por causa de uma queda · em vermelho, a falha; em destaque, o resultado



um arquivo, duas tentativas · efeito único

A queda no meio da tentativa 1 não exige nenhum tratamento especial: a tentativa 2 refaz tudo do início, e o banco descarta em silêncio o que já tinha sido gravado.

8.3 Tomando o arquivo com lease

A fila reentrega mensagens quando um worker some, mas pode também entregar a mesma mensagem a dois workers ao mesmo tempo (por exemplo, se o processamento passar do visibility timeout). O banco é o árbitro final, com um `UPDATE` atômico:

```
UPDATE arquivo
SET status          = 'PROCESSANDO',
    processando_ate = now() + interval '5 minutes',
    worker_id       = @worker_id,
    tentativas       = tentativas + 1
WHERE id = @arquivo_id
  AND (status = 'RECEBIDO'
      OR (status = 'PROCESSANDO' AND processando_ate < now()))
RETURNING tentativas;
```

Se nenhuma linha voltar, outro worker está com o arquivo (ou ele já terminou): descarte a mensagem. Se voltar, você é o dono pelos próximos 5 minutos. Como arquivos grandes demoram mais, o worker **renova o lease** periodicamente enquanto trabalha:

```
public sealed class RepositorioDeArquivos(IDbContextFactory<CobrancaDb>
    fabrica, IdentidadeDoWorker worker)
    : IRepositoryDeArquivos
{
    public async Task RenovarLeaseAsync(Guid arquivoId, CancellationToken ct)
    {
        await using CobrancaDb db = await fabrica.CreateDbContextAsync(ct);

        int afetadas = await db.Arquivos
            .Where(a => a.Id == arquivoId
                && a.WorkerId == worker.Id
                && a.Status == StatusArquivo.Processando)
            .ExecuteUpdateAsync(s => s.SetProperty(a => a.ProcessandoAte,
                DateTimeOffset.UtcNow.AddMinutes(5)), ct);

        if (afetadas == 0)
            throw new LeasePerdidoException(arquivoId); // alguém assumiu o
arquivo: pare tudo
    }

    // TentarAssumirAsync executa o UPDATE ... RETURNING acima; os demais
métodos seguem o mesmo padrão
}
```

O repositório é um singleton e cria um `DbContext` por operação a partir de `IDbContextFactory<CobrancaDb>` (Módulo 6), porque o worker que o usa também é singleton. O `IdentidadeDoWorker` é só um identificador estável do processo, como o nome do pod. O laço que chama `RenovarLeaseAsync` a cada minuto roda em paralelo com o pipeline do Módulo 5, ligado ao mesmo `CancellationTokenSource`, e está no projeto final: se o lease for perdido, o pipeline é cancelado. O mesmo raciocínio vale para a fila: no SQS, chame `ChangeMessageVisibility` periodicamente para estender o prazo da mensagem enquanto o arquivo é processado.

8.4 Retries do jeito certo

Retry só faz sentido para **erros transitórios**: aqueles em que a mesma operação, repetida um pouco depois, tem chance real de dar certo. Repetir um erro de layout é desperdício e atrasa a fila. A biblioteca padrão no .NET atual é o Polly v8, exposta também pelo pacote `Microsoft.Extensions.Resilience`:

```
builder.Services.AddResiliencePipeline("banco", pipeline =>
{
    pipeline
        .AddRetry(new RetryStrategyOptions
        {
            ShouldHandle = new PredicateBuilder()
                .Handle<NpgsqlException>(ex => ex.IsTransient)
                .Handle<TimeoutException>(),
            MaxRetryAttempts = 5,
            Delay = TimeSpan.FromMilliseconds(200),
            BackoffType = DelayBackoffType.Exponential,
            UseJitter = true
        })
        .AddTimeout(TimeSpan.FromSeconds(30));
});
```

Para aplicar o retry sem tocar no pipeline do Módulo 5, envolva o escritor num *decorator*: o pipeline continua recebendo um `IEscritorDeLotes` e nem sabe que existe retry.

```

public sealed class EscriitorResiliente(IEscriitorDeLotes interno,
ResiliencePipelineProvider<string> provedor)
    : IEscriitorDeLotes
{
    private readonly ResiliencePipeline _resiliencia =
provedor.GetPipeline("banco");

    public Task GravarAsync(Lote lote, CancellationToken ct) =>
        _resiliencia.ExecuteAsync(
            static async (estado, token) => await
estado.interno.GravarAsync(estado.lote, token),
            (interno, lote), ct).AsTask();
}

// Registro: o pipeline recebe o escritor já envolvido pelo retry
builder.Services.AddSingleton<EscriitorPostgres>();
builder.Services.AddSingleton<IEscriitorDeLotes>(sp => new EscriitorResiliente(
    sp.GetRequiredService<EscriitorPostgres>(),
    sp.GetRequiredService<ResiliencePipelineProvider<string>>()));

```

Por que cada peça existe:

- **Backoff exponencial** (200 ms, 400 ms, 800 ms...) dá tempo para o problema passar, por exemplo um failover do banco.
- **Jitter** (aleatoriedade no intervalo) evita que 50 workers que falharam ao mesmo tempo tentem de novo ao mesmo tempo, derrubando o banco que acabou de voltar. Sem jitter, o retry vira um ataque sincronizado.
- **IsTransient** é uma propriedade do Npgsql que já classifica erros de rede e alguns códigos do servidor como transitórios. Deadlocks (40P01) e falhas de serialização (40001) também merecem retry, desde que a transação inteira seja repetida (que é o caso aqui, porque cada tentativa chama GravarAsync do zero). Confira se a sua versão do Npgsql já os classifica como transitórios; se não, inclua-os no ShouldHandle comparando o SqlState da PostgresException.
- **Timeout externo** garante que uma tentativa travada não segure o escritor para sempre.

O retry só é seguro porque `GravarAsync` é uma transação idempotente. Esse é o elo entre os módulos 6 e 8.

8.5 Mensagens envenenadas e DLQ

Um arquivo que sempre derruba o worker (um bug de parsing que lança exceção não tratada, por exemplo) seria reentregue para sempre. Configure a fila para mover a mensagem para a DLQ depois de N recebimentos (no SQS, a `redrive policy` com

`maxReceiveCount`) e use o contador `tentativas` da tabela `arquivo` como segunda barreira: acima do limite, marque `FALHOU` e confirme a mensagem.

A DLQ precisa de dono e de processo: um alarme quando ela não está vazia, um painel que mostra o erro de cada arquivo e uma ação de "reenviar" para depois da correção. Uma DLQ que ninguém olha é só um lugar organizado para perder dados.

Regra de ouro da confirmação: **apague (ack) a mensagem só depois do commit final** do arquivo. Confirmar antes significa que um crash entre a confirmação e o commit perde o arquivo.

8.6 Outbox transacional

O problema do *dual write*: você precisa gravar no banco e publicar uma mensagem. Se gravar primeiro e o broker estiver fora, o evento se perde. Se publicar primeiro e o commit falhar, você anunciou algo que não aconteceu. Não existe transação distribuída prática entre os dois.

A outbox resolve gravando o evento como uma linha no mesmo banco, na mesma transação dos dados (já feito no Módulo 6). Um processo separado, o relay, publica:

```
public sealed class RelayDaOutbox(NpgsqlDataSource dataSource, IPublicador
publicador,
                                ILogger<RelayDaOutbox> log) :
    BackgroundService
{
    private const string Pegar = """
        SELECT id, tipo, chave, payload::text
        FROM outbox
        WHERE publicado_em IS NULL
        ORDER BY criado_em
        LIMIT 200
        FOR UPDATE SKIP LOCKED;
        """;

    protected override async Task ExecuteAsync(CancellationToken ct)
    {
        while (!ct.IsCancellationRequested)
        {
            int publicados = await PublicarLoteAsync(ct);
            if (publicados == 0)
                await Task.Delay(TimeSpan.FromMilliseconds(500), ct);
        }
    }

    private async Task<int> PublicarLoteAsync(CancellationToken ct)
```

```

{
    await using var conexao = await dataSource.OpenConnectionAsync(ct);
    await using var tx = await conexao.BeginTransactionAsync(ct);

    var eventos = new List<(Guid Id, string Tipo, string Chave, string
Payload)>();
    await using (var cmd = new NpgsqlCommand(Pegar, conexao, tx))
    await using (var reader = await cmd.ExecuteReaderAsync(ct))
        while (await reader.ReadAsync(ct))
            eventos.Add((reader.GetGuid(0), reader.GetString(1),
reader.GetString(2), reader.GetString(3)));

    if (eventos.Count == 0) return 0;

    await publicador.PublicarAsync(eventos, ct); // em lote, com o Id
como MessageId

    await using (var marcar = new NpgsqlCommand(
        "UPDATE outbox SET publicado_em = now() WHERE id = ANY(@ids)",
conexao, tx))
    {
        marcar.Parameters.AddWithValue("ids", eventos.Select(e =>
e.Id).ToArray());
        await marcar.ExecuteNonQueryAsync(ct);
    }

    await tx.CommitAsync(ct);
    return eventos.Count;
}
}

```

Pontos-chave:

- `FOR UPDATE SKIP LOCKED` permite várias réplicas do relay sem que duas peguem o mesmo evento.
- O relay publica **pelo menos uma vez**: se cair depois de publicar e antes do commit, o lote será publicado de novo. Por isso o `Id` do evento vai como identificador da mensagem, e os consumidores deduplicam.
- A ordem global não é garantida entre réplicas do relay. Se a ordem por chave importar, use uma fila com grupos de mensagens (SQS FIFO com `MessageGroupId` = chave, por exemplo) e uma única réplica de relay, ou particione o relay por chave.
- Limpe eventos publicados periodicamente (ou particione a outbox por dia e descarte partições antigas).

Alternativa em escala maior: em vez de consultar a tabela, ler o log de replicação lógica do PostgreSQL com uma ferramenta de CDC (Change Data Capture) como o Debezium. Reduz latência e carga de consulta, ao custo de mais infraestrutura.

8.7 Reconciliação: a última linha de defesa

Retries e idempotência protegem contra falhas que você previu. A reconciliação protege contra as que você não previu. Arquivos CNAB trazem no trailer a quantidade de registros e o somatório de valores; compare com o que de fato está no banco antes de marcar o arquivo como concluído:

```
SELECT count(*)           AS registros,
       coalesce(sum(valor_pago_centavos), 0) AS total_centavos
FROM retorno_detalhe
WHERE arquivo_id = @arquivo_id;
```

Se registros gravados + registros rejeitados for diferente do trailer, ou se o total divergir, o arquivo vai para **FALHOU** com o motivo "divergência de reconciliação" e dispara alerta. Essa verificação, barata e simples, já pegou mais bugs em sistemas reais do que qualquer teste unitário.

Os totais do trailer chegam até aqui pelo **IContextoDeLeitura** do Módulo 4. A implementação concreta guarda o que o leitor informou e os primeiros erros de linha (para o relatório do arquivo):

```
public sealed class ContextoDoArquivo(int limiteDeErros) : IContextoDeLeitura
{
    private readonly List<(int Linha, string Motivo)> _erros = [];

    public int LinhasLidas { get; private set; }
    public int LinhasRejeitadas { get; private set; }
    public int? QuantidadeNoTrailer { get; private set; }
    public long? TotalNoTrailerCentavos { get; private set; }
    public IReadOnlyList<(int Linha, string Motivo)> PrimeirosErros =>
        _erros;

    public void RegistrarErro(int numeroLinha, string motivo)
    {
        LinhasRejeitadas++;
        if (_erros.Count < limiteDeErros) _erros.Add((numeroLinha, motivo));
    }

    public void RegistrarTrailer(int quantidadeRegistros, long
        valorTotalCentavos)
        => (QuantidadeNoTrailer, TotalNoTrailerCentavos) =
```

```
(quantidadeRegistros, valorTotalCentavos);

    public void RegistrarFimDaLeitura(int linhasLidas) => LinhasLidas =
linhasLidas;
}
```

Ele não precisa ser thread-safe porque só o estágio de leitura escreve nele. A regra de reconciliação, então, fica assim:

1. Sem trailer (`QuantidadeNoTrailer` nulo): o arquivo está truncado. `FALHOU`, motivo "arquivo sem trailer".
2. Quantidade: registros gravados + linhas rejeitadas deve bater com o trailer. Em muitos layouts a quantidade do trailer inclui o próprio header e trailer; ajuste a comparação ao manual de cada banco, no perfil de layout.
3. Valor: a soma gravada deve bater com o total do trailer. Se houver linhas rejeitadas, o valor delas não está no banco e a soma não fecha; por isso, arquivo com rejeições não é comparado por valor e vai para revisão operacional, com o relatório de `PrimeirosErros`.

A consulta, a comparação e a mudança de status para `CONCLUIDO` ou `FALHOU` acontecem numa única transação, no método `ReconciliarEConcluirAsync` do repositório.

8.8 Desligamento gracioso

Quando o pod recebe SIGTERM: pare de buscar mensagens novas, deixe o pipeline atual terminar o lote em andamento (não o arquivo inteiro), não confirme a mensagem do arquivo incompleto e saia. A fila reentrega, o lease expira, outro worker assume e a idempotência cuida das linhas já gravadas. Opcionalmente, libere o lease explicitamente (`status = 'RECEBIDO'`) para que outro worker assuma na hora, sem esperar a expiração.

Exercício do Módulo 8

Faça um teste de caos. Processe um arquivo de 2 milhões de linhas com 2 réplicas do worker e, no meio, mate uma delas com `kill -9` (ou `docker kill`). Repita três vezes em momentos diferentes. Ao final, verifique: a contagem de `retorno_detalle` é exatamente igual ao trailer, não há dois eventos na outbox para a mesma linha, e o arquivo está `CONCLUIDO`. Depois derrube o banco por 30 segundos durante o processamento e confirme que os retries absorvem a queda sem enviar nada para a DLQ.

Módulo 9 — Observabilidade e medição de performance

Você só sabe que o pipeline atende a ficha do Módulo 1 se medir, em produção, quatro coisas: vazão (linhas por segundo), atraso (idade do arquivo mais antigo esperando), erros (rejeições, retries, DLQ) e saturação (canais cheios, pool de conexões esgotado, GC). O .NET 10 expõe tudo isso nativamente via `System.Diagnostics.Metrics` e `ActivitySource`, e o OpenTelemetry leva os dados para onde você quiser.

9.1 O que medir

Métrica	Tipo	Pergunta que responde
<code>arquivos.processados</code> (por status)	Counter	Quantos arquivos terminaram, e como?
<code>linhas.gravadas</code>	Counter	Qual a vazão real em linhas por segundo?
<code>arquivo.duracao</code>	Histogram (s)	Estamos dentro do SLA por arquivo? Qual o p95 e o p99?
<code>lote.duracao</code>	Histogram (ms)	O banco está ficando lento?
<code>fila.idade_mais_antiga</code>	Gauge (s)	Estamos atrasados? (melhor sinal para alerta e autoscaling)
<code>canal.ocupacao</code>	Gauge	Onde está o gargalo dentro do worker?
<code>retries</code> , <code>dlq.tamanho</code>	Counter, Gauge	O sistema está se recuperando sozinho ou acumulando falhas?
GC, ThreadPool, memória	Runtime	O processo está saudável?

A métrica de ocupação dos canais é o melhor diagnóstico interno que existe: se o canal de lotes está sempre cheio, o gargalo é o banco; se o canal de registros está sempre vazio, o gargalo é a leitura (rede ou parsing).

9.2 Métricas com System.Diagnostics.Metrics

```
using System.Diagnostics.Metrics;

public sealed class MetricasPipeline
{
    public const string NomeMeter = "Cobranca.Retorno";
```

```

private readonly Counter<long> _linhasGravadas;
private readonly Counter<long> _arquivosProcessados;
private readonly Histogram<double> _duracaoArquivo;
private readonly Histogram<double> _duracaoLote;

public MetricasPipeline(IMeterFactory fabrica)
{
    Meter meter = fabrica.Create(NomeMeter);
    _linhasGravadas = meter.CreateCounter<long>
("retorno.linhas.gravadas", unit: "{linha}");
    _arquivosProcessados = meter.CreateCounter<long>
("retorno.arquivos.processados", unit: "{arquivo}");
    _duracaoArquivo = meter.CreateHistogram<double>
("retorno.arquivo.duracao", unit: "s");
    _duracaoLote = meter.CreateHistogram<double>("retorno.lote.duracao",
unit: "ms");
}

public void LoteGravado(int linhas, double ms)
{
    _linhasGravadas.Add(linhas);
    _duracaoLote.Record(ms);
}

public void ArquivoFinalizado(string status, string banco, double
segundos)
{
    var tags = new TagList { { "status", status }, { "banco", banco } };
    _arquivosProcessados.Add(1, tags);
    _duracaoArquivo.Record(segundos, tags);
}
}

```

Registre com `services.AddSingleton<MetricasPipeline>()`. Use `IMeterFactory` em vez de `new Meter(...)` para que o DI controle o ciclo de vida e os testes possam inspecionar as medições.

Cuidado com a **cardinalidade das tags**: `status` e `banco` têm poucos valores possíveis, ótimo. Nunca use `arquivo_id` ou `nosso_numero` como tag: cada valor distinto cria uma série temporal nova e explode o custo do backend de métricas. Identificadores únicos vão em traces e logs, não em métricas.

9.3 Traces: a história de um arquivo

Um trace conta o caminho de um arquivo por todos os componentes. Crie um `ActivitySource` e abra um span para o arquivo e um span filho para cada etapa relevante:

```
public static class Telemetria
{
    public static readonly ActivitySource Fonte = new("Cobranca.Retorno");
}

using Activity? atividade = Telemetria.Fonte.StartActivity("processar-
arquivo", ActivityKind.Consumer);
atividade?.SetTag("arquivo.id", arquivoId);
atividade?.SetTag("arquivo.banco", banco);

// ... ao final
atividade?.SetTag("arquivo.linhas", totalLinhas);
atividade?.SetStatus(ActivityStatusCode.Ok);
```

Propague o contexto do trace pela fila: a ingestão grava o `traceparent` (padrão W3C) nos atributos da mensagem, e o worker continua o mesmo trace ao receber. Assim, no seu backend de observabilidade, você vê upload → fila → processamento → consolidação → publicação do evento como uma linha do tempo única. Para o volume de lotes, use amostragem (sampling) e evite criar um span por linha.

9.4 Ligando o OpenTelemetry

```
builder.Services.AddOpenTelemetry()
    .ConfigureResource(r => r.AddService("cobranca-retorno-worker"))
    .WithMetrics(m => m
        .AddMeter(MetricasPipeline.NomeMeter)
        .AddRuntimeInstrumentation() // GC, ThreadPool, memória
        .AddNpgsqlInstrumentation() // pool de conexões, duração de
comandos
    .WithTracing(t => t
        .AddSource("Cobranca.Retorno")
        .AddNpgsql()
        .AddHttpClientInstrumentation())
    .UseOtlpExporter(); // endpoint via
OTEL_EXPORTER_OTLP_ENDPOINT

builder.Logging.AddOpenTelemetry(l => l.IncludeScopes = true);
```

Os nomes dos métodos de extensão vêm de pacotes separados

(`OpenTelemetry.Instrumentation.Runtime`, `Npgsql.OpenTelemetry`, `OpenTelemetry.Instrumentation.Http`); confira as versões compatíveis com o seu Npgsql. O exportador OTLP é neutro: o mesmo código envia para um coletor que repassa para Datadog, Grafana, New Relic ou o serviço de observabilidade da sua nuvem.

9.5 Logs que ajudam, sem custar caro

Três regras para logs num pipeline de alto volume:

1. **Nunca logue por linha.** Um log por linha em um arquivo de 3 milhões de linhas é 3 milhões de eventos, mais caro que o processamento. Logue por arquivo e por lote, e agregue erros de linha ("312 linhas rejeitadas; primeiras 10: ...").
2. **Logs estruturados com o gerador de código.** O atributo `[LoggerMessage]` gera, em tempo de compilação, métodos de log sem boxing e sem formatação quando o nível está desligado:

```
public static partial class Logs
{
    [LoggerMessage(Level = LogLevel.Information,
        Message = "Arquivo {ArquivoId} concluído: {Linhas} linhas em {Segundos:F1}s")]
    public static partial void ArquivoConcluido(this ILogger logger, Guid
arquivoId, int linhas, double segundos);
}
```

3. **Correlacione:** com o OpenTelemetry ligado, cada log carrega o `TraceId` da atividade corrente, e você pula de um log para o trace completo do arquivo.

9.6 Ferramentas de diagnóstico

Ferramenta	Pergunta que responde	Uso típico
<code>dotnet-counters</code>	Como estão CPU, GC, ThreadPool e minhas métricas agora?	<code>dotnet-counters monitor -n Worker --counters System.Runtime,Cobranca.Retorno</code>
<code>dotnet-trace</code>	Onde o tempo de CPU está sendo gasto?	Coletar 30 s sob carga e abrir no PerfView ou no speedscope
<code>dotnet-gcdump</code>	Que objetos estão ocupando o heap?	Investigar crescimento de memória
<code>dotnet-dump</code>	O que cada thread estava fazendo?	Processo travado ou deadlock

Ferramenta	Pergunta que responde	Uso típico
<code>pg_stat_statements</code>	Quais consultas consomem mais tempo no banco?	Ranking por tempo total e por chamada

Em Kubernetes, as ferramentas `dotnet-*` rodam num container auxiliar (ephemeral container ou sidecar) que compartilha o namespace de processos com o worker.

9.7 Micro-benchmarks com BenchmarkDotNet

Para decisões de hot path (qual parser, qual tamanho de buffer), use BenchmarkDotNet, que cuida de warmup, iterações e análise estatística:

```
[MemoryDiagnoser]
public class ParserBenchmark
{
    private byte[][] _linhas = default!;

    [GlobalSetup]
    public void Setup() => _linhas = GeradorDeLinhas.Gerar(quantidade:
10_000);

    [Benchmark(Baseline = true)]
    public long ComSubstring()
    {
        long soma = 0;
        foreach (var bytes in _linhas)
        {
            string linha = Encoding.Latin1.GetString(bytes);
            soma += long.Parse(linha.Substring(253, 13));
        }
        return soma;
    }

    [Benchmark]
    public long ComSpan()
    {
        long soma = 0;
        foreach (var bytes in _linhas)
            if (Cnab400Parser.TryParseDetalhe(bytes, 0, out var r, out _))
                soma += r.ValorPagoCentavos;
        return soma;
    }
}
```

Rode sempre em `Release`, fora do depurador. A coluna `Allocated` do `MemoryDiagnoser` é tão importante quanto o tempo: menos alocação significa menos GC em produção, onde o benchmark não enxerga.

9.8 Teste de carga ponta a ponta

Micro-benchmarks não mostram gargalos de banco nem de concorrência. Monte um teste de carga com o ambiente completo (Docker Compose com PostgreSQL, um emulador de S3 e de fila) e um gerador de arquivos sintéticos que reproduza a distribuição real: muitos arquivos pequenos, alguns grandes, um gigante. Aumente a carga em degraus e anote, para cada degrau, linhas por segundo, p95 da duração de lote e CPU do banco. O ponto em que a vazão para de subir enquanto a latência dispara é a sua capacidade real. Compare com a meta da ficha do Módulo 1.

9.9 Alertas que importam

Alerte sobre sintomas que afetam o negócio, não sobre cada causa possível:

- Arquivo mais antigo na fila há mais do que metade do SLA (ainda dá tempo de reagir).
- DLQ com qualquer mensagem.
- Qualquer divergência de reconciliação.
- Nenhum arquivo processado numa janela em que normalmente chegam muitos (o pipeline pode estar parado sem erro).

Exercício do Módulo 9

Instrumente o pipeline com as métricas da tabela 9.1, suba o .NET Aspire Dashboard em Docker (ele recebe OTLP e mostra métricas, traces e logs localmente) e rode o teste de carga. Encontre, só pelas métricas, qual estágio é o gargalo. Depois mude o número de escritores e confirme a mudança na ocupação dos canais.

Módulo 10 — Escala horizontal e deploy

Workers stateless escalam adicionando réplicas, e o sinal certo para escalar é o **atraso da fila**, não a CPU. O teto dessa escala não é o Kubernetes: é o número de conexões e a capacidade de escrita do banco, e o deploy precisa respeitar isso.

10.1 Por que não escalar por CPU

O autoscaler padrão do Kubernetes (HPA) olha CPU e memória. Um worker de arquivos passa boa parte do tempo esperando o banco, então a CPU fica baixa mesmo com a fila crescendo. O HPA conclui que está tudo bem enquanto o SLA estoura. O sinal correto é o trabalho pendente: mensagens na fila, ou melhor ainda, a idade da mensagem mais antiga.

10.2 Autoscaling por fila com KEDA

O KEDA (Kubernetes Event-Driven Autoscaling) lê métricas de filas e alimenta o HPA. Para SQS:

```
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: retorno-worker
spec:
  scaleTargetRef:
    name: retorno-worker          # o Deployment do worker
  minReplicaCount: 1
  maxReplicaCount: 8             # teto calculado pelo banco (10.3)
  pollingInterval: 15
  cooldownPeriod: 300            # espera 5 min sem carga antes de reduzir
  triggers:
    - type: aws-sqs-queue
      authenticationRef:
        name: keda-aws-credenciais
      metadata:
        queueURL: https://sqs.sa-east-1.amazonaws.com/<conta>/retornos
        awsRegion: sa-east-1
        queueLength: "4"          # alvo: 4 mensagens por réplica
```

Com `queueLength: "4"`, 20 arquivos na fila geram 5 réplicas. Para arquivos de tamanhos muito diferentes, essa conta engana: 4 arquivos gigantes valem mais que 40 pequenos. Uma opção mais precisa é publicar uma métrica própria (linhas pendentes, ou idade do arquivo mais antigo) e usar o scaler de Prometheus ou de métricas externas do KEDA.

`minReplicaCount: 0` (escalar a zero) economiza recursos fora das janelas de pico, mas adiciona o tempo de subir um pod ao primeiro arquivo do dia. Para SLA de 15 minutos isso é aceitável; para SLA de segundos, mantenha pelo menos uma réplica.

10.3 O teto de escala é o banco

Cada réplica abre até (arquivos em paralelo × escritores) conexões de escrita, mais algumas para lease e metadados. O total precisa caber no orçamento de conexões que o banco reserva para esse sistema:

$$R_{\max} = \left\lfloor \frac{C}{A \cdot E + X} \right\rfloor$$

C é o número de conexões que o banco reserva para o pipeline, A é quantos arquivos cada réplica processa em paralelo, E é o número de escritores por arquivo e X são as conexões extras por réplica (lease, metadados).

Exemplo: o banco reserva 80 conexões para o pipeline; cada réplica processa 2 arquivos com 4 escritores e usa 2 conexões extras. São 10 por réplica, logo no máximo 8 réplicas. Esse é o `maxReplicaCount`. Escalar além disso não aumenta a vazão; só cria espera no pool ou erros de "too many connections".

Mas lembre do Módulo 5: mesmo dentro do orçamento, existe um ponto em que mais escritores reduzem a vazão. O teto real é o menor entre o limite de conexões e o ponto de saturação medido no teste de carga.

10.4 O Deployment do worker

```
spec:
  template:
    spec:
      terminationGracePeriodSeconds: 60    # maior que o ShutdownTimeout (45
s)
      containers:
        - name: worker
          image: <registro>/cobranca-retorno-worker:1.4.2
          resources:
            requests: { cpu: "1", memory: 1Gi }
            limits:   { memory: 1Gi }        # requests = limits de memória
          env:
            - name: DOTNET_gcServer
              value: "1"
          livenessProbe:
            httpGet: { path: /health/live, port: 8080 }
            periodSeconds: 30
            failureThreshold: 4
```

Decisões explicadas:

- **Memória com requests igual a limits.** O pod recebe garantia do que pode usar, e o GC trabalha com um teto previsível (75% de 1 GiB para o heap). Sem limite de CPU, o worker pode usar folga do nó sem sofrer throttling, que costuma piorar a latência.
- **Liveness com cuidado.** Um worker no meio de um lote de 30 segundos não pode ser morto por "não responder". A sonda deve verificar se o laço principal fez progresso recentemente (um timestamp atualizado a cada lote ou a cada busca na fila), com tolerância folgada, e nunca depender do banco: banco fora do ar não se resolve reiniciando o worker.
- **Readiness** quase não importa para workers, porque ninguém chama o pod; ele consome.
- **PodDisruptionBudget** com `maxUnavailable: 1` evita que manutenções do cluster derrubem todas as réplicas de uma vez.

A verificação de progresso em código:

```
public sealed class BatimentoDoWorker
{
    private long _ultimoTick = Environment.TickCount64;
    public void Marcar() => Interlocked.Exchange(ref _ultimoTick,
Environment.TickCount64);
    public TimeSpan DesdeUltimo =>
TimeSpan.FromMilliseconds(Environment.TickCount64 - Interlocked.Read(ref
_ultimoTick));
}

public sealed class WorkerVivoCheck(BatimentoDoWorker batimento) :
IHealthCheck
{
    public Task<HealthCheckResult> CheckHealthAsync(HealthCheckContext
context, CancellationToken ct = default)
        => Task.FromResult(batimento.DesdeUltimo < TimeSpan.FromMinutes(3)
        ? HealthCheckResult.Healthy()
        : HealthCheckResult.Unhealthy($"sem progresso há
{batimento.DesdeUltimo}"));
}
```

Como o Worker Service não tem servidor HTTP, exponha o endpoint com um `WebApplication` mínimo (Kestrel só para `/health` e, se usar Prometheus, `/metrics`), ou troque a sonda por um comando que verifica um arquivo de batimento.

10.5 Imagens de container

O SDK do .NET publica imagens sem Dockerfile: `dotnet publish -c Release /t:PublishContainer`. Configure no `.csproj` a imagem base (`ContainerBaseImage`), o nome e as tags. Pontos importantes:

- As imagens oficiais rodam como usuário não-root (`app`) por padrão desde o .NET 8. Mantenha assim.
- Imagens **chiseled** (Ubuntu sem shell nem gerenciador de pacotes) reduzem tamanho e superfície de ataque. A desvantagem é não ter shell para depuração; use ephemeral containers para isso.
- **ReadyToRun** (`<PublishReadyToRun>true</PublishReadyToRun>`) pré-compila o código e reduz o tempo de subida, o que ajuda quando o KEDA escala de 0 ou de 1 para 8 de repente.
- **Native AOT** gera um binário nativo com subida quase instantânea e memória menor, mas exige código sem reflexão dinâmica. O Npgsql tem caminho compatível (`NpgsqlSlimDataSourceBuilder`), enquanto o suporte do EF Core a AOT ainda tem

limitações. Recomendação: worker principal com JIT + ReadyToRun (o Dynamic PGO do JIT costuma dar mais vazão em processos longos); considere AOT para serviços pequenos como a ingestão.

10.6 Particionando o trabalho

Quando uma única fila e um único pool de workers não bastam, separe por característica:

- **Fila rápida e fila pesada:** a ingestão roteia pelo tamanho do arquivo. Arquivos pequenos nunca esperam atrás de um gigante, e cada fila tem seu próprio `ScaledObject` e limites.
- **Por origem** (banco, cedente): isola falhas. Um banco que manda arquivos corrompidos só enche a própria DLQ.
- **Por prioridade de negócio:** liquidações antes de relatórios informativos.

Cada partição recebe uma fatia explícita do orçamento de conexões do banco.

10.7 Recuperação de desastre e custo

Como todo processamento pode ser refeito a partir do storage, a estratégia de recuperação fica simples: replique o bucket (replicação entre regiões), mantenha backups do banco com recuperação para um ponto no tempo e saiba reenfileirar os arquivos de um período. Escreva e teste esse procedimento antes de precisar dele.

Para custo, os três itens que crescem sem ninguém perceber: arquivos originais no storage (use regras de ciclo de vida para mover para classes mais baratas depois de N dias, respeitando a retenção regulatória), volume de logs (Módulo 9) e armazenamento do banco (particionamento e retenção, Módulo 7).

Exercício do Módulo 10

Num cluster local (kind ou minikube) com KEDA instalado e uma fila local (LocalStack para SQS, ou RabbitMQ com o scaler correspondente), faça o deploy do worker com o `ScaledObject`. Publique 200 arquivos de uma vez e observe as réplicas subindo até o máximo e voltando depois do `cooldownPeriod`. Durante o pico, faça um rollout de nova versão e confirme que nenhum arquivo foi perdido ou duplicado.

Projeto final — o pipeline completo, passo a passo

O projeto final junta as peças dos dez módulos num sistema que roda localmente com Docker Compose e no Kubernetes, e que passa nos critérios de aceite da ficha do Módulo 1. Ele é feito em onze etapas, cada uma terminando com algo que funciona e tem teste.

Estrutura da solução

```

Cobranca.Retorno.sln
├─ src/
│   ├─ Cobranca.Retorno.Dominio/      registros, status do arquivo, regras (sem
dependências)
│   ├─ Cobranca.Retorno.Layouts/      parsers CNAB 240/400 em span, perfis por
banco
│   ├─ Cobranca.Retorno.Infra/        Npgsql, EF Core, S3, fila, outbox,
resiliência
│   ├─ Cobranca.Retorno.Ingestao/     recebe arquivos, calcula hash, publica na
fila
│   ├─ Cobranca.Retorno.Worker/       pipeline com Channels, lease,
reconciliação
│   └─ Cobranca.Retorno.Relay/        publica eventos da outbox
├─ tests/
│   ├─ Cobranca.Retorno.Layouts.Tests/ testes de unidade com arquivos de
referência
│   ├─ Cobranca.Retorno.Integracao.Tests/ Testcontainers (PostgreSQL,
LocalStack)
│   └─ Cobranca.Retorno.Benchmarks/   BenchmarkDotNet
└─ deploy/
    └─ docker-compose.yml             PostgreSQL, LocalStack, Aspire
Dashboard
    └─ k8s/                          Deployments, ScaledObjects, PDBs
  
```

A separação em `Layouts` isolado é proposital: o parser é o código mais testado e mais sensível do sistema, não depende de banco nem de rede e pode ser exercitado com milhares de arquivos de exemplo em segundos.

As onze etapas

- ☐ **1. Fundação.** Solução, `docker-compose` com PostgreSQL e LocalStack, migrations das tabelas `arquivo`, `arquivo_chunk`, `retorno_detalhe` e `outbox`. Critério: `dotnet test` sobe os containers e aplica as migrations do zero.
- ☐ **2. Ingestão.** Upload para o S3 com hash calculado em streaming, `INSERT` com `UNIQUE` no hash, publicação na fila só depois do registro. Critério: enviar o mesmo arquivo duas vezes gera um registro e uma mensagem.
- ☐ **3. Parser.** `Cnab400Parser` e `LeitorCnab400` dos Módulos 3 e 4, com header, trailer e `IContextoDeLeitura` recebendo erros de linha e os totais do trailer. Critério: testes com arquivos de referência (válido, linha curta, data inválida, trailer divergente, última linha sem quebra, linha cruzando borda do buffer).
- ☐ **4. Pipeline em memória.** `PipelineDeArquivo` do Módulo 5 com escritor falso. Critério: memória estável com escritor lento; erro em um estágio cancela os outros.

- ☐ **5. Escritor real.** `EscreitorPostgres` do Módulo 6 com `COPY` e consolidação. Critério: 1 milhão de linhas gravadas; regravar o mesmo lote não muda contagens nem gera eventos.
- ☐ **6. Tomada e lease.** `UPDATE` condicional, renovação com `PeriodicTimer`, limite de tentativas. Critério: dois workers recebendo a mesma mensagem, só um processa.
- ☐ **7. Resiliência.** Pipeline Polly no escritor, DLQ configurada na fila, classificação de erros transitórios e permanentes. Critério: banco derrubado por 30 segundos não gera DLQ; arquivo com layout inválido vai para `REJEITADO` na primeira tentativa.
- ☐ **8. Relay da outbox.** `RelayDaOutbox` do Módulo 8 e um consumidor de teste com `inbox`. Critério: matar o relay no meio de um lote não duplica efeitos no consumidor.
- ☐ **9. Reconciliação e conclusão.** Comparação com o trailer, transição para `CONCLUIDO` ou `FALHOU`, confirmação da mensagem só depois disso. Critério: um bug proposital que descarta uma linha é detectado.
- ☐ **10. Observabilidade.** Métricas, traces e logs do Módulo 9 no Aspire Dashboard. Critério: identificar o gargalo só pelas métricas durante o teste de carga.
- ☐ **11. Carga, caos e deploy.** Teste de carga em degraus, teste de caos com `kill -9`, deploy no Kubernetes com KEDA. Critério: atender a vazão e o SLA da ficha com margem, e sobreviver ao rollout durante o pico.

Extensão opcional, depois da etapa 9: o fan-out em chunks do Módulo 2.4, se o maior arquivo previsto não couber no SLA com um único worker.

Como testar o worker

- **Unidade:** o parser contra arquivos de referência guardados em `tests/` (um arquivo por caso da etapa 3), e o `ContextoDoArquivo` com as regras de reconciliação.
- **Integração:** Testcontainers com PostgreSQL e LocalStack (S3 e SQS). O teste sobe o host do worker dentro do processo de teste (`Host.CreateApplicationBuilder` apontando para os containers), envia um arquivo, publica a mensagem e espera, com prazo, o status `CONCLUIDO`; depois confere contagens em `retorno_detalhe` e eventos na `outbox`.
- **Idempotência:** o mesmo teste processa o mesmo arquivo duas vezes e verifica que nada mudou na segunda.
- **Caos e carga:** a etapa 11, fora do CI de pull request (num job agendado), comparando a vazão com a medição anterior para pegar regressões.

Contratos e registro de dependências

O laço principal depende de três abstrações. Elas isolam a fila, o storage e o banco, e são justamente o que os testes de integração trocam ou apontam para containers:

```

public sealed record MensagemArquivo(Guid ArquivoId, string ChaveStorage,
string Banco, string IdRecebimento);

public interface IFilaDeArquivos
{
    IAsyncEnumerable<MensagemArquivo> ConsumirAsync(CancellationTokens ct);
    Task ConfirmarAsync(MensagemArquivo mensagem, CancellationTokens ct);
    // ack: apaga a mensagem
    Task EstenderVisibilidadeAsync(MensagemArquivo mensagem, TimeSpan prazo,
CancellationTokens ct);
}

public interface IArmazenamento
{
    Task<Stream> AbrirLeituraAsync(string chave, CancellationTokens ct);
}

public interface IRepositorioDeArquivos
{
    Task<int?> TentarAssumirAsync(Guid arquivoId, CancellationTokens ct);
    // UPDATE condicional (Módulo 8.3)
    Task RenovarLeaseAsync(Guid arquivoId, CancellationTokens ct);
    // lança LeasePerdidoException
    Task<StatusArquivo> ReconciliarEConcluirAsync(Guid arquivoId,
ContextoDoArquivo contexto, CancellationTokens ct);
    Task MarcarRejeitadoAsync(Guid arquivoId, string motivo,
CancellationTokens ct);
    Task MarcarFalhaAsync(Guid arquivoId, string motivo, CancellationTokens
ct);
}

```

O `IdRecebimento` é o identificador que a fila exige para confirmar ou estender a mensagem (o *receipt handle* do SQS). O registro no host deixa explícito o tempo de vida de cada peça; todas são singletons, porque o worker é singleton e nenhuma guarda estado de requisição:

```

var builder = Host.CreateApplicationBuilder(args);

builder.Services.AddNpgsqlDataSource(builder.Configuration.GetConnectionString("Cobranca")!);
builder.Services.AddDbContextFactory<CobrancaDb>((sp, o) => o
    .UseNpgsql(sp.GetRequiredService<NpgsqlDataSource>())
    .UseSnakeCaseNamingConvention());

builder.Services.AddResiliencePipeline("banco", p => { /* Módulo 8.4 */ });

```

```

builder.Services.AddSingleton<EscritorPostgres>();
builder.Services.AddSingleton<IEscritorDeLotes>(sp => new EscritorResiliente(
    sp.GetRequiredService<EscritorPostgres>(),
    sp.GetRequiredService<ResiliencePipelineProvider<string>>()));

builder.Services.AddSingleton<LeitorCnab400>();
builder.Services.AddSingleton<PipelineDeArquivo>();
builder.Services.AddSingleton<IdentidadeDoWorker>();
builder.Services.AddSingleton<IRepositorioDeArquivos, RepositorioDeArquivos>
    ();
builder.Services.AddSingleton<IArmazenamento, ArmazenamentoS3>();
builder.Services.AddSingleton<IFilaDeArquivos, FilaSqs>();
builder.Services.AddSingleton<BatimentoDoWorker>();
builder.Services.AddSingleton<MetricasPipeline>();

builder.Services.AddOptions<ProcessamentoOptions>()

.BindConfiguration("Processamento").ValidateDataAnnotations().ValidateOnStart
    ();
builder.Services.AddHostedService<ProcessadorDeArquivosWorker>();

await builder.Build().RunAsync();

```

O host já valida escopos e a construção do grafo de dependências em `Development`; mantenha essa validação ligada nos testes de integração para que uma dependência cativa futura quebre o build, não a produção.

O laço principal do worker

Este é o código que amarra tudo. Cada linha corresponde a uma decisão tomada em algum módulo; os comentários apontam qual.

```

public sealed class ProcessadorDeArquivosWorker(
    IFilaDeArquivos fila,
    IRepositorioDeArquivos arquivos,
    IArmazenamento storage,
    PipelineDeArquivo pipeline,
    BatimentoDoWorker batimento,
    MetricasPipeline metricas,
    ILogger<ProcessadorDeArquivosWorker> log) : BackgroundService
{
    private const int MaxTentativas = 5;

    protected override async Task ExecuteAsync(CancellationTokens ct)
    {
        await Task.Yield(); // libera o host para terminar de iniciar (Módulo

```

3)

```

        await Parallel.ForEachAsync(                                // concorrência
limitada (Módulo 5)
            fila.ConsumirAsync(ct),
            new ParallelOptions { MaxDegreeOfParallelism = 2,
CancellationToken = ct },
            ProcessarMensagemAsync);
    }

    private async ValueTask ProcessarMensagemAsync(MensagemArquivo msg,
CancellationToken ct)
    {
        batimento.Marcar();
        using Activity? atividade =
Telemetria.Fonte.StartActivity("processar-arquivo", ActivityKind.Consumer);
        atividade?.SetTag("arquivo.id", msg.ArquivoId);
        long inicio = Stopwatch.GetTimestamp();

        int? tentativa = await arquivos.TentarAssumirAsync(msg.ArquivoId,
ct); // lease (Módulo 8)
        if (tentativa is null)
        {
            await fila.ConfirmarAsync(msg, ct); // outro worker está com ele,
ou já terminou
            return;
        }

        if (tentativa > MaxTentativas)
        {
            await arquivos.MarcarFalhaAsync(msg.ArquivoId, "tentativas
esgotadas", ct);
            await fila.ConfirmarAsync(msg, ct);
            return;
        }

        using var cts = CancellationTokenSource.CreateLinkedTokenSource(ct);
        Task lease = ManterLeaseAsync(msg, cts);

        try
        {
            var contexto = new ContextoDoArquivo(limiteDeErros: 1_000);
// Módulo 8.7
            await using Stream origem = await
storage.AbrirLeituraAsync(msg.ChaveStorage, cts.Token); // stream (Módulo 4)

```

```

        await pipeline.ProcessarAsync(msg.ArquivoId, origem, contexto,
cts.Token);          // Channels + COPY (Módulos 5 e 6)

        StatusArquivo final = await
arquivos.ReconciliarEConcluirAsync(msg.ArquivoId, contexto, cts.Token);
        await fila.ConfirmarAsync(msg, ct); // ack só depois do commit
final

        double segundos = Stopwatch.GetElapsedTime(inicio).TotalSeconds;
        metricas.ArquivoFinalizado(final.ToString(), msg.Banco,
segundos);          // Módulo 9
        log.ArquivoConcluido(msg.ArquivoId, contexto.LinhasLidas,
segundos);
    }
    catch (LayoutInvalidoException ex)
    {
        await arquivos.MarcasRejeitadoAsync(msg.ArquivoId, ex.Message,
ct); // erro permanente: sem retry
        await fila.ConfirmarAsync(msg, ct);
    }
    catch (OperationCanceledException) when (ct.IsCancellationRequested)
    {
        // SIGTERM: não confirma; a fila reentrega e a idempotência cuida
do resto (Módulo 8)
    }
    catch (OperationCanceledException)
    {
        log.LogWarning("Processamento de {ArquivoId} interrompido: lease
perdido", msg.ArquivoId);
        // não confirma: quem assumiu o arquivo termina o trabalho
    }
    catch (Exception ex)
    {
        log.LogError(ex, "Falha ao processar {ArquivoId}",
msg.ArquivoId);
        // não confirma: a fila reentrega; após N recebimentos, vai para
a DLQ
    }
    finally
    {
        await cts.CancelAsync();
        await lease;          // ManterLeaseAsync nunca lança: trata os
próprios erros
        batimento.Marcas();
    }
}

```

```

    private async Task ManterLeaseAsync(MensagemArquivo msg,
CancellationTokensource cts)
    {
        try
        {
            using var timer = new PeriodicTimer(TimeSpan.FromMinutes(1));
            while (await timer.WaitForNextTickAsync(cts.Token))
            {
                await arquivos.RenovarLeaseAsync(msg.ArquivoId, cts.Token);
                await fila.EstenderVisibilidadeAsync(msg,
TimeSpan.FromMinutes(5), cts.Token);
                batimento.Marcas();
            }
        }
        catch (OperationCanceledException)
        {
            // fim normal: o processamento terminou e cancelou o lease
        }
        catch (Exception ex)
        {
            // lease perdido ou banco indisponível: não é seguro continuar
sem a posse do arquivo
            log.LogWarning(ex, "Lease de {ArquivoId} não pôde ser mantido",
msg.ArquivoId);
            await cts.CancelAsync();
        }
    }
}

```

Repare que o worker não tem nenhum `try/catch` dentro do pipeline nem retry no laço principal. Cada responsabilidade está num lugar só: retries transitórios no escritor (Polly), reentrega na fila, duplicatas no banco. O laço principal só decide **quando confirmar a mensagem**, que é a decisão que mais importa. A `LayoutInvalidoException`, lançada pelo leitor quando o header não corresponde a nenhum perfil de layout conhecido, é o único erro de arquivo que dispensa retry: repetir não muda o conteúdo.

Crerios de aceite finais

O projeto está pronto quando estas afirmações forem verdadeiras e houver um teste ou uma medição que prove cada uma:

- ☐ Vazão sustentada $\geq$ meta de pico da ficha, com 2× de margem
- ☐ O maior arquivo previsto termina dentro do SLA
- ☐ Memória de cada réplica estável durante o processamento do maior arquivo

- ☐ Reprocessar qualquer arquivo não altera contagens nem gera eventos
- ☐ `kill -9` em qualquer momento não perde nem duplica registros
- ☐ Toda divergência com o trailer gera alerta
- ☐ Um arquivo inválido não atrasa os demais
- ☐ O gargalo é identificável pelas métricas, sem depurador

Referências e próximos passos

Depois do projeto final, os caminhos naturais de aprofundamento são três: internals do PostgreSQL (MVCC, WAL, planejador), gerenciamento de memória no .NET e padrões de sistemas distribuídos. Quando o .NET 11 sair, vale testar o pipeline nele e medir a diferença com o BenchmarkDotNet antes de decidir migrar.

A outra metade do sistema, a API que recebe os arquivos e responde consultas sobre as mesmas tabelas, está no [curso de Minimal APIs no ASP.NET Core 10](#). Os dois compartilham o projeto `Cobranca.Dados`, a tabela `arquivo` e o padrão de outbox.

Documentação oficial por módulo

Módulo	Leitura
3. Runtime	Fundamentos do GC no .NET · Memória e spans
4. I/O	System.IO.Pipelines
5. Concorrência	System.Threading.Channels
6. Escrita	Npgsql: COPY · PostgreSQL: INSERT e ON CONFLICT · PostgreSQL: carga em massa
7. Leitura	PostgreSQL: EXPLAIN · PostgreSQL: particionamento
8. Confiabilidade	Polly · Padrão outbox transacional
9. Observabilidade	Diagnóstico no .NET · BenchmarkDotNet
10. Escala	KEDA: scaler de AWS SQS

Livros

- *Designing Data-Intensive Applications*, Martin Kleppmann: a base teórica de filas, replicação, idempotência e processamento em lote. Leia junto com os Módulos 2 e 8.

- *Pro .NET Memory Management*, Konrad Kokosa: o GC por dentro. Complementa o Módulo 3.
- *PostgreSQL Internals*, Egor Rogov (gratuito, publicado pela Postgres Professional): MVCC, WAL, índices e planejador. Complementa os Módulos 6 e 7.

Fontes sobre versões

- [.NET 11 RC1 Gets Go-Live Support Ahead of November Launch](#) (Visual Studio Magazine, 14/09/2026): RC1 com go-live desde 8 de setembro, C# 15 com unions, lançamento previsto para a .NET Conf a partir de 10 de novembro de 2026.
- [.NET 11 RC1, .NET 10.0.12, 9.0.20 e 8.0.31](#) (atualizações de 8 de setembro de 2026).